

MYSQL 고급

SELECT Additional Option

Ch 4.

Additional SELECT

4.1 | 함수^{Function}

4.2 | Additional SELECT Option

Objective

- ❖ SELECT 구문에서 함수를 사용할 수 있다
- ❖ SELECT 구문에서 ORDER BY option을 사용할 수 있다
- ❖ SELECT 구문에서 GROUP BY option을 사용할 수 있다
- ❖ SELECT 구문에서 HAVING option을 사용할 수 있다
- ❖ 하나 이상의 테이블로부터 데이터를 조회할 수 있다
- ❖ 두 개의 SELECT 구문 실행 결과를 하나로 결합할 수 있다
- ❖ SELECT 구문에서 서브쿼리^{Subquery} option을 사용할 수 있다

Ch 4.

Additional SELECT

4.1 | 함수 Function

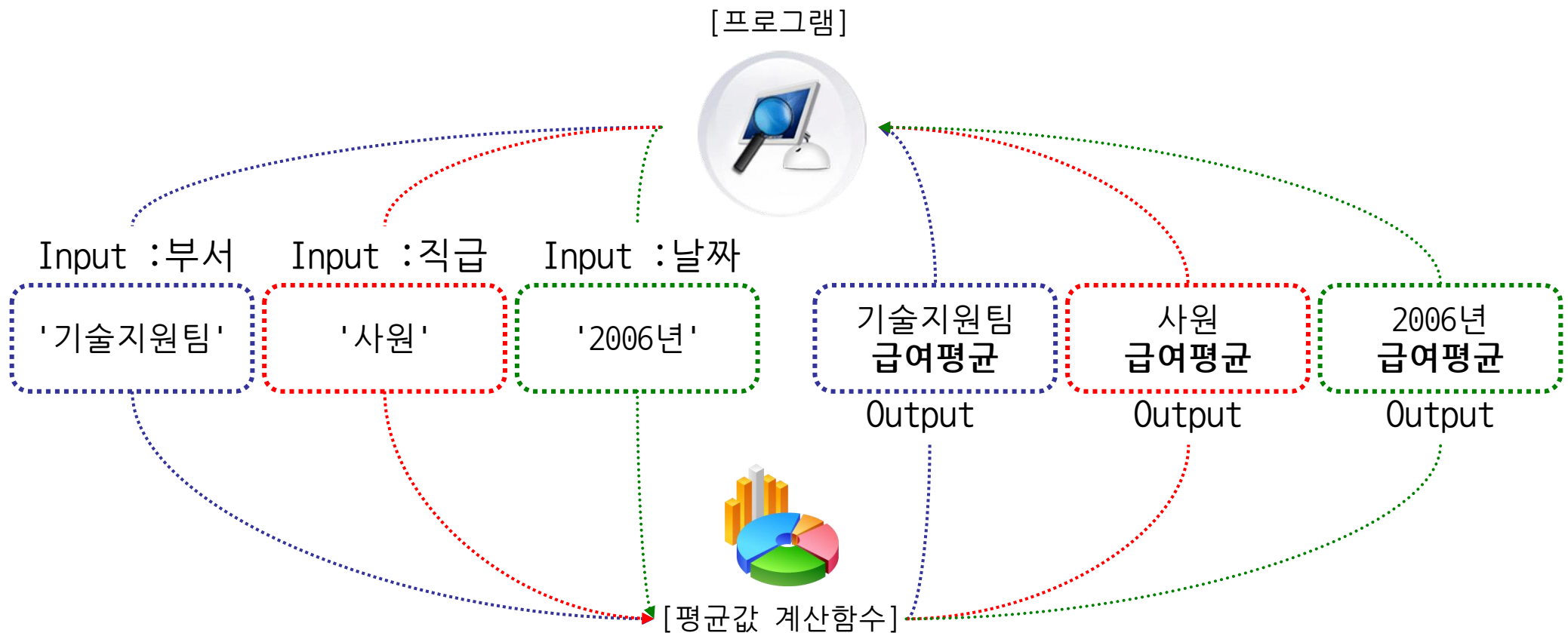
단일 행 함수 Single-Row Function

그룹 함수 Group Function

4.2 | Additional SELECT Option

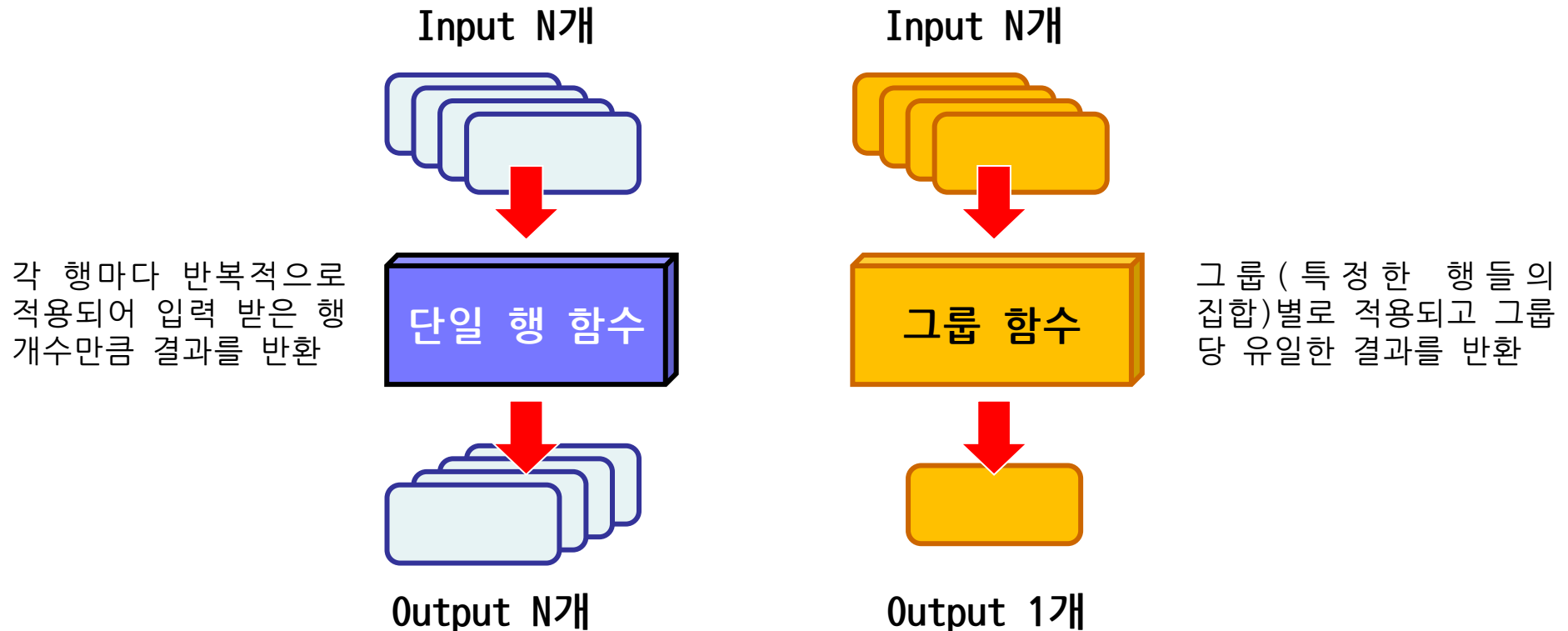
4.1.1 함수 개념

- 하나의 큰 프로그램에서 반복적으로 사용되는 부분들을 분리한 작은 서브 프로그램
- 호출^{call}하고, 실행 결과를 리턴^{return} 하는 방식으로 사용



4.1.2 함수 유형

반환 결과에 따라 단일 행 함수^{Single-Row Function} 와 그룹 함수^{Group Function} 로 구분



4.1.2 함수 유형 예

직원 테이블(일부)

사번	성별	근무지	나이
001	M	마포구 상암동 상암IT센터	29
002	M	중구 회현동 프라임 타워	30
003	F	영등포구 문래동 강서빌딩	25
004	M	영등포구 여의도동 LG 트윈타워	27
005	F	강남구 역삼동 GS타워	24

사번	근무지
001	마포구
002	중구
003	영등포구
004	영등포구
005	강남구

단일 행 함수
사용 결과

성별	평균나이
M	28.6
F	24.5

그룹 함수
사용 결과

4.1.3 주요 단일 행 함수

구분	입력 값 타입	리턴 값 타입	종류
문자(열) 함수	CHARACTER	CHARACTER	LPAD/RPAD, LTRIM/RTRIM/TRIM, SUBSTR
		NUMBER	INSTR, LENGTH/LENGTHB
숫자 함수	NUMBER	NUMBER	ROUND, TRUNC
날짜 함수	DATE	DATE	ADD_MONTHS, SYSDATE
		NUMBER	MONTHS_BETWEEN
타입변환 함수	ANY	ANY	TO_CHAR, TO_DATE, TO_NUMBER
기타 함수	ANY	ANY	NVL, DECODE

주오라클 전용 함수 중 일부만 표시

4.1.3 주요 단일 행 함수 - 문자열 함수 LENGTH

주어진 컬럼 값/문자열 길이(문자 개수)를 반환하는 함수

[구문]

[반환 타입]

[파라미터]

LENGTH(*string*)

NUMBER

CHARACTER 타입의 컬럼 또는 임의의 문자열

[구문 특징]

- CHAR 데이터 타입 컬럼 값을 입력 받은 경우
실제 데이터 길이(문자 개수)에 상관없이 컬럼 전체 길이(문자 개수)를 반환
- VARCHAR2 데이터 타입 컬럼 값을 입력 받은 경우
실제 데이터 길이(문자 개수) 반환

4.1.3 주요 단일 행 함수 - 문자열 함수 LENGTH 사용 예

[COLUMN_LENGTH 테이블]

Columns of COLUMN_LENGTH	
Name	Type
CHARTYPE	CHAR(20)
VARCHARTYPE	VARCHAR2(20)

[샘플 데이터]

CHARTYPE	VARCHARTYPE
LG CNS	LG CNS
엘지씨엔에스	엘지씨엔에스

[적용 예]

```
SELECT LENGTH(CHARTYPE),
       LENGTH(VARCHARTYPE)
FROM   COLUMN_LENGTH;
```

LENGTH(CHARTYPE)	LENGTH(VARCHARTYPE)
20	6
14	6

CHAR 타입 컬럼에 한글이 포함되었을 때 LENGTH 함수 적용 결과

- 한글 1문자=2바이트, 6문자이므로 $6 \times 2 = 12$ 바이트
- 컬럼 길이(20바이트) - 데이터 길이(12바이트) = 8바이트
- 데이터 6문자 + 여유 공간 8문자(영문자 기준으로) = 14

참고 오라클 DBMS를 설치할 때 어떤 설정을 하느냐에 따라 한글 1문자의 바이트 수가 달라짐
 (본 과정에서는 Oracle 10g K016MSWIN949 character set 사용 → 한글 1문자는 2바이트로 처리됨)

4.1.3 주요 단일 행 함수 - 문자열 함수 LENGTHB

주어진 컬럼 값/문자열 길이(Byte)를 반환하는 함수

[구문]

[반환 타입]

[파라미터]

LENGTHB(*string*)

NUMBER

CHARACTER 타입의 컬럼 또는 임의의 문자열

[구문 특징]

- Single-byte Character Set 경우
LENGTH 함수 반환 값과 동일(문자 개수 = 바이트 수)
- Multi-byte Character Set 경우
Character Set에 따라 동일한 데이터에 대한 바이트 처리 결과가 다름

4.1.3 주요 단일 행 함수 - 문자열 함수 LENGTHB 사용 예

[COLUMN_LENGTH 테이블]

Columns of COLUMN_LENGTH	
Name	Type
CHARTYPE	CHAR(20)
VARCHARTYPE	VARCHAR2(20)

[샘플 데이터]

CHARTYPE	VARCHARTYPE
LG CNS	LG CNS
엘지씨엔에스	엘지씨엔에스

[적용 예]

```
SELECT LENGTHB(CHARTYPE),
       LENGTHB(VARCHARTYPE)
FROM   COLUMN_LENGTH;
```

LENGTHB(CHARTYPE)	LENGTHB(VARCHARTYPE)
20	6
20	12

4.1.3 주요 단일 행 함수 - 문자열 함수 INSTR

찾는 문자(열)이 지정한 위치부터 지정한 회수만큼 나타난 시작 위치를 반환하는 함수

[구문]

[반환 타입]

```
INSTR(string, substring, [ position, [ occurrence ] ] )
```

NUMBER

[파라미터]

파라미터	설명
string	문자 타입 컬럼 또는 문자열
substring	찾으려는 문자(열)
position	• 어디부터 찾을지를 결정하는 시작 위치(기본값 1) • POSITION > 0 : STRING의 시작부터 끝 방향으로 찾는 의미 • POSITION < 0 : STRING의 끝부터 시작 방향으로 찾는 의미
occurrence	• SUBSTRING이 반복될 때의 지정하는 빈도(기본값 1) • 음수 값은 사용할 수 없음

4.1.3 주요 단일 행 함수 - 문자열 함수 INSTR 사용 예

EMAIL 컬럼 값의 '@vcc.com' 문자열 중 "." 바로 앞의 문자 'c' 위치를 구하시오

[구문 1]

```
SELECT EMAIL,
       INSTR( EMAIL, 'c', -1, 2 ) 위치
FROM   EMPLOYEE ;
```

'POSITION'은 음수(-1)이므로 끝부터 시작 방향으로 찾음
'OCCURRENCE'는 2 이므로 두 번째 나타나는 문자 위치를 의미

[구문 2]

```
SELECT EMAIL,
       INSTR( EMAIL, 'c', INSTR( EMAIL, '.' )-1 ) 위치
FROM   EMPLOYEE ;
```

'POSITION'은 INSTR(EMAIL, '.')-1 결과(≡ 마침표 한 자리 앞 의미)
'OCCURRENCE'는 생략되었으므로 기본 값 1 적용(첫 번째 나타나는 문자 의미)

[결과(일부)]

EMAIL	위치
sg_ahn@vcc.com	10
jh_park@vcc.com	11
ms_choi@vcc.com	11
sy_kang@vcc.com	11
sg_han@vcc.com	10
jh_jo@vcc.com	9
jih_jeon@vcc.com	12
hs_kim@vcc.com	10
sw_cha@vcc.com	10
sm_kim@vcc.com	10
hg_song@vcc.com	11
ws_jeong@vcc.com	12
jh_um@vcc.com	9
hk_shin@vcc.com	11
sw_jo@vcc.com	9
hi_park@vcc.com	11
sw_kwon@vcc.com	11

4.1.3 주요 단일 행 함수 - 문자열 함수 LPAD/RPAD

주어진 컬럼/문자열에 임의의 문자(열)을 왼쪽/오른쪽에 덧붙여 길이 N의 문자열을 반환하는 함수

[구문]

[반환 타입]

LPAD(*string*, *N*, [*str*])
 RPAD(*string*, *N*, [*str*])

CHARACTER

[파라미터]

파라미터	설명
string	문자 타입 컬럼 또는 문자열
N	- 반환할 문자(열)의 길이(바이트) - 원래 STRING 길이보다 작다면 N만큼만 잘라서 표시
str	- 덧붙이려는 문자(열) - 생략하면 공백 한 문자

4.1.3 주요 단일 행 함수 - 문자열 함수 LPAD 사용 예

EMAIL 컬럼 왼쪽에 '.' 을 덧붙여 길이를 20으로 맞추시오

```
SELECT EMAIL AS 원본데이터,
       LENGTH(EMAIL) AS 원본길이,
       LPAD(EMAIL, 20, '.') AS 적용결과,
       LENGTH(LPAD(EMAIL, 20, '.')) AS 결과길이
FROM   EMPLOYEE;
```

[결과(일부)]



원본데이터	원본길이	적용결과	결과길이
sg_ahn@vcc.com	14	sg_ahn@vcc.com	20
jh_park@vcc.com	15	jh_park@vcc.com	20
ms_choi@vcc.com	15	ms_choi@vcc.com	20
sy_kang@vcc.com	15	sy_kang@vcc.com	20
sg_han@vcc.com	14	sg_han@vcc.com	20
jh_jo@vcc.com	13	jh_jo@vcc.com	20
jih_jeon@vcc.com	16	jih_jeon@vcc.com	20

오른쪽 정렬 효과를 나타냄

4.1.3 주요 단일 행 함수 - 문자열 함수 RPAD 사용 예

EMAIL 컬럼 오른쪽에 '.' 을 덧붙여 길이를 20으로 맞추시오

```
SELECT EMAIL AS 원본데이터,  
       LENGTH(EMAIL) AS 원본길이,  
       RPAD(EMAIL, 20, '.') AS 적용결과,  
       LENGTH(RPAD(EMAIL, 20, '.')) AS 결과길이  
FROM   EMPLOYEE;
```

[결과(일부)]



원본데이터	원본길이	적용결과	결과길이
sg_ahn@vcc.com	14	sg_ahn@vcc.com.....	20
jh_park@vcc.com	15	jh_park@vcc.com.....	20
ms_choi@vcc.com	15	ms_choi@vcc.com.....	20
sy_kang@vcc.com	15	sy_kang@vcc.com.....	20
sg_han@vcc.com	14	sg_han@vcc.com.....	20
jh_jo@vcc.com	13	jh_jo@vcc.com.....	20
jih_jeon@vcc.com	16	jih_jeon@vcc.com....	20

왼쪽 정렬 효과를 나타냄

4.1.3 주요 단일 행 함수 - 문자열 함수 LTRIM/RTRIM

- 주어진 컬럼/문자열의 왼쪽/오른쪽에서 지정한 STR에 포함된 모든 문자를 제거한 나머지를 반환하는 함수
- 패턴을 제거하는 의미가 아님

[구문]

[반환 타입]

LTRIM(*string*, *str*)
 RTRIM(*string*, *str*)

CHARACTER

[파라미터]

파라미터	설명
string	문자 타입 컬럼 또는 문자열
str	- 제거하려는 문자(열) - 생략하면 왼쪽/오른쪽 공백을 모두 제거

4.1.3 주요 단일 행 함수 - 문자열 함수 LTRIM 사용 예

^주 SELECT LTRIM('...tech') FROM DUAL;	tech
SELECT LTRIM('...tech', '.') FROM DUAL;	tech
SELECT LTRIM('000123', '0') FROM DUAL;	123
SELECT LTRIM('123123Tech', '123') FROM DUAL;	Tech
SELECT LTRIM('123123Tech123', '123') FROM DUAL;	Tech123
SELECT LTRIM('xyxzyyyTech', 'xyz') FROM DUAL;	Tech
SELECT LTRIM('6372Tech', '0123456789') FROM DUAL;	Tech

^주 SELECT ... FROM DUAL 구문은 테스트 용도로 사용하는 구문임

4.1.3 주요 단일 행 함수 - 문자열 함수 RTRIM 사용 예

SELECT RTRIM('tech...') FROM DUAL;	tech
SELECT RTRIM('tech...', '.') FROM DUAL;	tech
SELECT RTRIM('123000', '0') FROM DUAL;	123
SELECT RTRIM('Tech123123', '123') FROM DUAL;	Tech
SELECT RTRIM('123Tech123', '123') FROM DUAL;	123Tech
SELECT RTRIM('Techxyxzyyy', 'xyz') FROM DUAL;	Tech
SELECT RTRIM('Tech6372', '0123456789') FROM DUAL;	Tech

4.1.3 주요 단일 행 함수 - 문자열 함수 TRIM

주어진 컬럼/문자열의 앞/뒤/양쪽에 있는 지정한 문자를 제거한 나머지를 반환하는 함수

[구문]

[반환 타입]

```
TRIM( trim_source )
TRIM( trim_char FROM trim_source )
TRIM( LEADING|TRAILING|BOTH [trim_char] FROM trim_source )
```

CHARACTER

[파라미터]

파라미터	설명
trim_source	문자 타입 컬럼 또는 문자열
trim_char	- 제거하려는 <u>하나의 문자</u> - 생략하면 기본값으로 공백 한 문자
LEADING TRAILING BOTH	- trim_char 위치 지정 - 앞/뒤/양쪽 지정 가능(기본값은 양쪽)

4.1.3 주요 단일 행 함수 - 문자열 함수 TRIM 사용 예

SELECT TRIM('··tech··') FROM DUAL;	tech
SELECT TRIM('a' FROM 'aatechaaa') FROM DUAL;	tech
SELECT TRIM(LEADING '0' FROM '000123') FROM DUAL;	123
SELECT TRIM(TRAILING '1' FROM 'Tech1') FROM DUAL;	Tech
SELECT TRIM(BOTH '1' FROM '123Tech111') FROM DUAL;	23Tech
SELECT TRIM(LEADING FROM '··Tech··') FROM DUAL;	Tech··

4.1.3 주요 단일 행 함수 - 문자열 함수 SUBSTR

주어진 컬럼/문자열에서 지정한 위치부터 지정한 개수 만큼의 문자열을 잘라내어 반환하는 함수

[구문]

[반환 타입]

SUBSTR(*string*, *position*, [*length*])

CHARACTER

[파라미터]

파라미터	설명
string	문자 타입 컬럼 또는 문자열
position	• 잘라내는 시작 위치 • position = 0 or 1 : 시작 위치(1번째) • position > 0 : 끝 방향으로 지정한 수만큼 위치 • position < 0 : 시작 방향으로 지정한 수만큼 위치
length	• 반환할 문자 개수 • 잘라내는 방향은 항상 끝 방향 • 생략하면 position부터 문자열 끝까지를 의미 • length < 0 : NULL 반환

4.1.3 주요 단일 행 함수 - 문자열 함수 SUBSTR 사용 예

SELECT SUBSTR('This·is·a·test', 6, 2) FROM DUAL;	is
SELECT SUBSTR('This·is·a·test', 6) FROM DUAL;	is·a·test
SELECT SUBSTR('이것은·연습입니다', 3, 4) FROM DUAL;	은·연습
SELECT SUBSTR('TechOnTheNet', 1, 4) FROM DUAL;	Tech
SELECT SUBSTR('TechOnTheNet', -3, 3) FROM DUAL;	Net
SELECT SUBSTR('TechOnTheNet', -6, 3) FROM DUAL;	The
SELECT SUBSTR('TechOnTheNet', -8, 2) FROM DUAL;	On

4.1.3 주요 단일 행 함수 - 숫자 함수 ROUND

지정한 자릿수에서 반올림 하는 함수

[구문]

[반환 타입]

```
ROUND( number, [decimal_places] )
```

NUMBER

[파라미터]

파라미터	설명
number	숫자 타입 컬럼 또는 임의 숫자
decimal_places	- 반올림되어 표시되는 소수점 자리 - 반드시 정수 값 사용 - 생략되면 0 의미 - decimal_places > 0 : 소수점 이하 자리 의미 - decimal_places < 0 : 소수점 이상 자리 의미

4.1.3 주요 단일 행 함수 - 숫자 함수 ROUND 사용 예

SELECT ROUND(125.315) FROM DUAL;	125
SELECT ROUND(125.315, 0) FROM DUAL;	125
SELECT ROUND(125.315, 1) FROM DUAL;	125.3
SELECT ROUND(125.315, -1) FROM DUAL;	130
SELECT ROUND(125.315, 3) FROM DUAL;	125.315
SELECT ROUND(-125.315, 2) FROM DUAL;	-125.32

4.1.3 주요 단일 행 함수 - 숫자 함수 TRUNC

지정한 자릿수에서 버림 하는 함수

[구문]

[반환 타입]

```
TRUNC( number, [decimal_places] )
```

NUMBER

[파라미터]

파라미터	설명
number	숫자 타입 컬럼 또는 임의 숫자
decimal_places	• 버림 결과가 표시되는 소수점 자리 • 반드시 정수 값 사용 • 생략되면 0 의미 • decimal_places > 0 : 소수점 이하 자리 의미 • decimal_places < 0 : 소수점 이상 자리 의미

4.1.3 주요 단일 행 함수 - 숫자 함수 TRUNC 사용 예

SELECT TRUNC(125.315) FROM DUAL;	125
SELECT TRUNC(125.315, 0) FROM DUAL;	125
SELECT TRUNC(125.315, 1) FROM DUAL;	125.3
SELECT TRUNC(125.315, -1) FROM DUAL;	120
SELECT TRUNC(125.315, 3) FROM DUAL;	125.315
SELECT TRUNC(-125.315, -3) FROM DUAL;	0

4.1.3 주요 단일 행 함수 - 날짜 함수 SYSDATE

- 지정된 형식으로 현재 날짜와 시간을 표시하는 함수
- 한글 버전으로 설치된 경우 기본 설정 형식은 '년도(2자리)/월(2자리)/일(2자리)'

[구문]

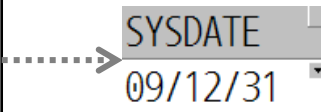
SYSDATE

[반환 타입]

DATE

[사용 예]

```
SELECT SYSDATE  
FROM DUAL;
```



SYSDATE
09/12/31

4.1.3 주요 단일 행 함수 - 날짜 함수 ADD_MONTHS

지정한 만큼의 달 수를 더한 날짜를 반환하는 함수

[구문]

[반환 타입]

ADD_MONTHS(*date*, *N*)

DATE

[파라미터]

파라미터	설명
date	기준이 되는 날짜
N	date에 더하려는 월 수

4.1.3 주요 단일 행 함수 - 날짜 함수 ADD_MONTHS 사용 예

직원 별로 입사일 기준으로 근무한 지 20년이 되는 일자를 조회하시오.

```
SELECT EMP_NAME,
       HIRE_DATE,
       ADD_MONTHS( HIRE_DATE, 240 )
FROM   EMPLOYEE;
```

EMP_NAME	HIRE_DATE	ADD_MONTHS(HIRE_DATE,240)
한선기	90/04/01	10/04/01
강중훈	04/04/30	24/04/30
최만식	95/12/30	15/12/30
정도연	97/06/03	17/06/03
안석규	98/07/01	18/07/01
조재형	98/11/23	18/11/23
정지현	04/07/15	24/07/15
김예수	01/03/20	21/03/20
나승원	01/03/20	21/03/20
김순이	99/10/20	19/10/20
성해교	03/08/16	23/08/16
전우성	02/07/14	22/07/14
엄정하	04/07/21	24/07/21
심하균	04/09/30	24/09/30
고승우	03/04/11	23/04/11
박하일	04/11/10	24/11/10
권상후	01/05/20	21/05/20
임영애	00/01/31	20/01/31
염정하	03/09/17	23/09/17
김솔오	96/10/01	16/10/01
이중기	04/10/01	24/10/01
감우섭	05/07/31	25/07/31

4.1.3 주요 단일 행 함수 - 날짜 함수 MONTHS_BETWEEN

지정한 두 날짜 사이의 월 수를 반환하는 함수

[구문]

[반환 타입]

MONTHS_BETWEEN(*date1*, *date2*)

NUMBER

[파라미터]

파라미터	설명
<i>date1</i> <i>date2</i>	· <i>date1</i>, <i>date2</i> 는 날짜 · <i>date1</i> > <i>date2</i> : 양수 반환 · <i>date1</i> < <i>date2</i> : 음수 반환

날짜가 크다는 의미는 더 나중 날짜라는 의미임

4.1.3 주요 단일 행 함수 - 날짜 함수 MONTHS_BETWEEN 사용 예

SELECT MONTHS_BETWEEN('09/01/01', '09/03/14') FROM DUAL;	-2.41935483870968
SELECT MONTHS_BETWEEN('09/07/01', '09/03/14') FROM DUAL;	3.58064516129032
SELECT MONTHS_BETWEEN('09/03/01', '09/03/01') FROM DUAL;	0
SELECT MONTHS_BETWEEN('09/08/02', '09/06/02') FROM DUAL;	2

결과 중 정수가 아닌 부분은 달의 일부를 의미함

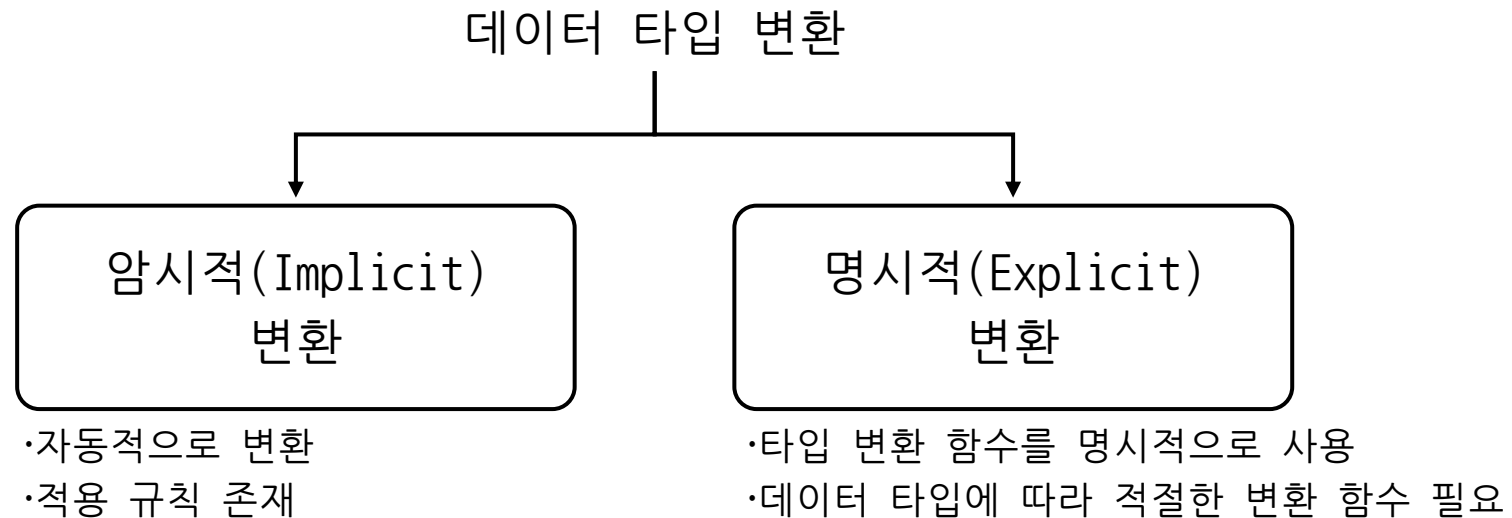
'2010년 1월 1일' 기준으로 입사한지 10년이 넘는 직원들의 근무년수 조회

```
SELECT EMP_NAME,
       HIRE_DATE,
       MONTHS_BETWEEN('10/01/01', HIRE_DATE)/12 AS 근무년수
FROM   EMPLOYEE
WHERE  MONTHS_BETWEEN('10/01/01', HIRE_DATE) > 120;
```

EMP_NAME	HIRE_DATE	근무년수
최만식	95/12/30	14.005376344086
정도연	97/06/03	12.5779569892473
안석규	98/07/01	11.5
조재형	98/11/23	11.1075268817204
김순이	99/10/20	10.1989247311828
김술오	96/10/01	13.25

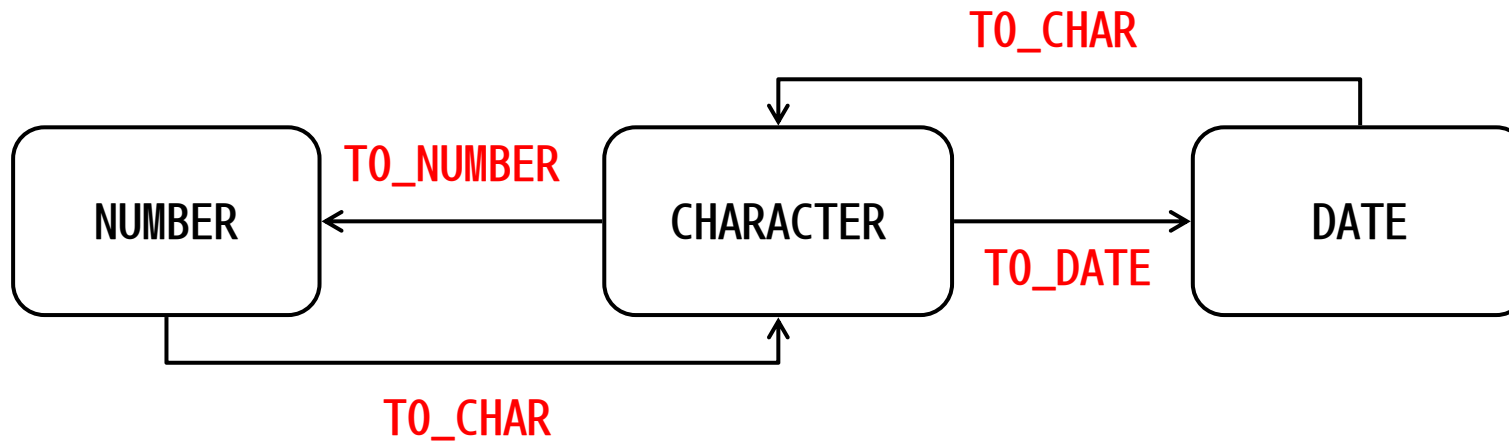
4.1.3 주요 단일 행 함수 - 데이터 타입 변환

오라클 DBMS는 데이터 타입을 변환하는 두 가지 방법을 제공



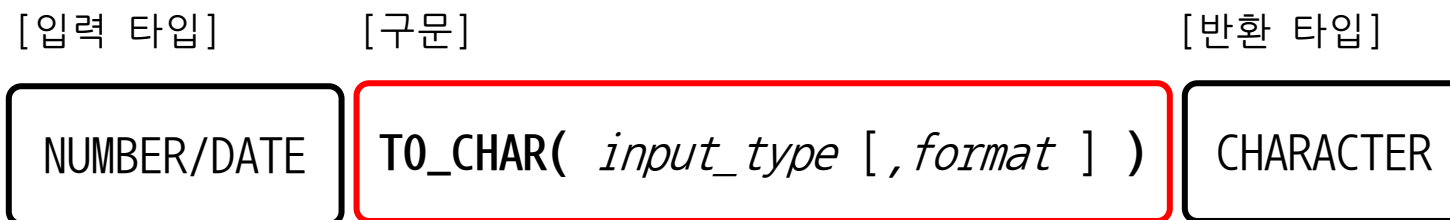
4.1.3 주요 단일 행 함수 - 데이터 타입 변환

- 데이터 타입에 따라 사용할 수 있는 변환 함수가 다름
- 상호 타입 변환이 되지 않는 데이터 타입 존재



4.1.3 주요 단일 행 함수 - 데이터 타입 변환 함수 TO_CHAR

NUMBER/DATE 타입을 CHARACTER 타입으로 변환하는 함수



[함수 설명]

- NUMBER 타입을 CHARACTER 타입으로 변환이 필요한 경우
 - 표현 형식을 변경 : 1000 → 1,000
 - 숫자를 문자로 사용 : 100 → '100'
- DATE 타입을 CHARACTER 타입으로 변환이 필요한 경우
 - "년,월,일" 표현 형식을 변경 : 09/12/30 → 2009-12-30
 - 시간 정보 표시
 - CHARACTER 타입과 비교 : HIRE_DATE = '03/06/14'
- 'format' 파라미터는 변경하려는 표현 형식을 위해 제공되는 규칙

4.1.3 주요 단일 행 함수 - 데이터 타입 변환 함수 TO_CHAR 사용 예

사번이 100인 직원 이름과 급여 조회

- CHAR 타입과 NUMBER 타입 비교는 불가능
- 내부적으로 CHAR 타입인 EMP_ID 값이 NUMBER 타입으로 변환되어 처리됨 → 암시적 변환

```
SELECT EMP_NAME,  
       SALARY  
FROM   EMPLOYEE  
WHERE  EMP_ID = 100;
```

TO_CHAR 함수를 사용하여
NUMBER 타입을 CHAR 타입으로
변환 후 비교 → 명시적 변환

```
SELECT EMP_NAME,  
       SALARY  
FROM   EMPLOYEE  
WHERE  EMP_ID = TO_CHAR(100);
```



EMP_NAME	SALARY
한선기	9000000

4.1.3 주요 단일 행 함수 - 데이터 타입 변환 함수 TO_CHAR 사용 예

[제공되는 숫자 표현형식]

형식	설명
9	자리 수 지정
0	남는 자리를 0으로 표시
\$ 또는 L	통화기호 표시
. 또는 ,	지정한 위치에 . 또는 , 표시
EEEE	과학 지수 표기법

SELECT TO_CHAR(1234, '99999') FROM DUAL;	1234
SELECT TO_CHAR(1234, '09999') FROM DUAL;	01234
SELECT TO_CHAR(1234, 'L99999') FROM DUAL;	₩1234
SELECT TO_CHAR(1234, '99,999') FROM DUAL;	1,234
SELECT TO_CHAR(1234, '09,999') FROM DUAL;	01,234
SELECT TO_CHAR(1000, '9.9EEEE') FROM DUAL;	1.0E+03
SELECT TO_CHAR(1234, '999') FROM DUAL;	###

4.1.3 주요 단일 행 함수 - 데이터 타입 변환 함수 TO_CHAR 사용 예

[제공되는 날짜 표현형식]

형식	설명
YYYY/YY/YEAR	년도(4자리숫자/뒤 2자리숫자/문자)
MONTH/MON/MM/RM	달(이름/약어/숫자/로마 기호)
DDD/DD/D	일(1년 기준/1달 기준/1주 기준)
Q	분기(1, 2, 3, 4)
DAY/DY	요일(이름/약어 이름)
HH(12)/HH24	12시간/24시간 표시
AM PM	오전, 오후
MI	분(0~59)
SS	초(0~59)

4.1.3 주요 단일 행 함수 - 데이터 타입 변환 함수 TO_CHAR 사용 예

SELECT TO_CHAR(SYSDATE, 'PM HH24:MI:SS') FROM DUAL;	오후 20:57:11
SELECT TO_CHAR(SYSDATE, 'AM HH:MI:SS') FROM DUAL;	오후 08:57:11
SELECT TO_CHAR(SYSDATE, 'MON DY, YYYY') FROM DUAL;	1월 월, 2010
SELECT TO_CHAR(SYSDATE, 'YYYY-fmMM-DD DAY') FROM DUAL;	2010-1-4 월요일
SELECT TO_CHAR(SYSDATE, 'YYYY-MM-fmDD DAY') FROM DUAL;	2010-01-4 월요일
SELECT TO_CHAR(SYSDATE, 'Year, Q') FROM DUAL;	Twenty Ten, 1

'fm' 모델을 사용하면 '01' 형식이 '1' 형식으로 표현됨

4.1.3 주요 단일 행 함수 - 데이터 타입 변환 함수 TO_CHAR 사용 예

```
SELECT EMP_NAME AS 이름,
       TO_CHAR(HIRE_DATE, 'YYYY-MM-DD') AS 입사일
FROM   EMPLOYEE
WHERE  JOB_ID = 'J7';
```

이름	입사일
정지현	2004-07-15
성해교	2003-08-16
고승우	2003-04-11
염정하	2003-09-17

```
SELECT EMP_NAME AS 이름,
       TO_CHAR(HIRE_DATE, 'YYYY"년" MM"월" DD"일"') AS 입사일
FROM   EMPLOYEE
WHERE  JOB_ID = 'J7';
```

이름	입사일
정지현	2004년 07월 15일
성해교	2003년 08월 16일
고승우	2003년 04월 11일
염정하	2003년 09월 17일

```
SELECT EMP_NAME AS 이름,
       SUBSTR(HIRE_DATE,1,2)||'년 '||
       SUBSTR(HIRE_DATE,4,2)||'월 '||
       SUBSTR(HIRE_DATE,7,2)||'일' AS 입사일
FROM   EMPLOYEE
WHERE  JOB_ID = 'J7';
```

이름	입사일
정지현	04년 07월 15일
성해교	03년 08월 16일
고승우	03년 04월 11일
염정하	03년 09월 17일

4.1.3 주요 단일 행 함수 - 데이터 타입 변환 함수 TO_CHAR 사용 예

DATE 타입과 CHARACTER 타입 비교

```

SELECT EMP_NAME AS 이름,
       HIRE_DATE AS 기본입사일,
       TO_CHAR(HIRE_DATE,
               'YYYY/MM/DD HH24:MI:SS') AS 상세입사일
FROM   EMPLOYEE
WHERE  JOB_ID IN ('J1','J2');

```

이름	기본입사일	상세입사일
한선기	90/04/01	2090/04/01 13:30:30
강중훈	04/04/30	2004/04/30 00:00:00
최만식	95/12/30	1995/12/30 00:00:00

DATE 타입에 시간 정보 없이 날짜만 입력하면 자동으로 시간은 해당 날짜의 '00시 00분 00초'로 저장

```

SELECT EMP_NAME
FROM   EMPLOYEE
WHERE  HIRE_DATE = '04/04/30';

```

EMP_NAME
강중훈

시간 정보가 없는 경우에는 기본 날짜 형식으로 비교 가능

```

SELECT EMP_NAME
FROM   EMPLOYEE
WHERE  HIRE_DATE = '90/04/01';

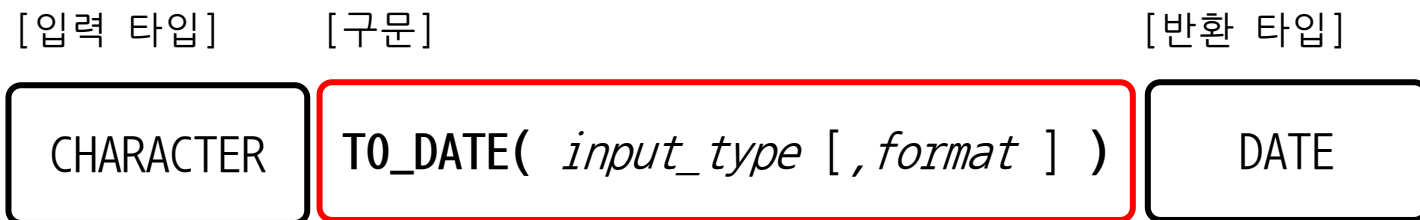
```

EMP_NAME

시간 정보가 있는 경우에는 기본 날짜 형식으로는 비교 불가능

4.1.3 주요 단일 행 함수 - 데이터 타입 변환 함수 TO_DATE

CHARACTER 타입을 DATE 타입으로 변환하는 함수



[함수 설명]

- 기본 날짜 형식이 아닌 문자열을 DATE 타입으로 인식시켜야 하는 경우
 - 2010-01-04 : 기본 날짜 형식이 아니므로 문자열로 인식
- DATE 타입과 비교하는 경우
 - HIRE_DATE = '90/04/01 13:30:30'
- 제공되는 날짜 표현 형식을 이용

4.1.3 주요 단일 행 함수 - 데이터 타입 변환 함수 TO_DATE 사용 예

SELECT TO_DATE('20100101', 'YYYYMMDD') FROM DUAL;	10/01/01
SELECT TO_CHAR('20100101', 'YYYY, MON') FROM DUAL;	N/A(오류)
SELECT TO_CHAR(TO_DATE('20100101', 'YYYYMMDD'), 'YYYY, MON') FROM DUAL;	2010, 1월
SELECT TO_DATE('041030 143000', 'YYMMDD HH24MISS') FROM DUAL;	04/10/30
SELECT TO_CHAR(TO_DATE('041030 143000', 'YYMMDD HH24MISS'), 'DD-MON-YY HH:MI:SS PM') FROM DUAL;	30-10월-04 02:30:00 오후
SELECT TO_DATE('980630', 'YYMMDD') FROM DUAL;	98/06/30
SELECT TO_CHAR(TO_DATE('980630', 'YYMMDD'), 'YYYY.MM.DD') FROM DUAL;	2098.06.30

4.1.3 주요 단일 행 함수 - 데이터 타입 변환 함수 TO_DATE 사용 예

```
SELECT EMP_NAME,  
       HIRE_DATE  
FROM   EMPLOYEE  
WHERE  HIRE_DATE =  
       TO_DATE('900401 133030','YYMMDD HH24MISS');
```

```
SELECT EMP_NAME,  
       HIRE_DATE  
FROM   EMPLOYEE  
WHERE  TO_CHAR(HIRE_DATE, 'YYMMDD') = '900401';
```



EMP_NAME	HIRE_DATE
한선기	90/04/01

4.1.3 주요 단일 행 함수 - 데이터 타입 변환 함수 TO_DATE

RR 날짜 형식

- YY 날짜 형식과 유사
- '지정한 년도'와 '현재 년도'에 따라 반환하는 '세기' 값이 달라짐 → 여러 세기 지정 가능

		지정한 년도(두 자리)	
		0 - 49	50 - 99
현재 년도 (두 자리)	0 - 49	현재 세기 반환	이전 세기 반환
	50 - 99	다음 세기 반환	현재 세기 반환

```
SELECT EMP_NAME,
       HIRE_DATE,
       TO_CHAR(HIRE_DATE, 'YYYY/MM/DD')
FROM   EMPLOYEE
WHERE  EMP_NAME = '한선기';
```

EMP_NAME	HIRE_DATE	TO_CHAR(HIRE_DATE, 'YYYY/MM/DD')
한선기	90/04/01	2090/04/01

- 해당 데이터는 두 자리 년도를 YY 형식으로 사용
TO_DATE('90/04/01 13:30:30', 'YY/MM/DD HH24:MI:SS')
- YY 형식은 항상 현재 세기를 의미
→ 원래 의미 '1990' 대신 현재 세기인 '2090'으로 인식

4.1.3 주요 단일 행 함수 - 데이터 타입 변환 함수 TO_DATE 사용 예

현재 년도	지정 날짜	RR 형식	YY 형식
1995	95/10/27	1995	1995
1995	17/10/27	2017	1917
2009	17/10/27	2017	2017
2009	95/10/27	1995	2095

```

SELECT '2009/10/14' AS 현재,
       '95/10/27' AS 입력,
       TO_CHAR(TO_DATE('95/10/27', 'YY/MM/DD'), 'YYYY/MM/DD') AS YY형식1,
       TO_CHAR(TO_DATE('95/10/27', 'YY/MM/DD'), 'RRRR/MM/DD') AS YY형식2,
       TO_CHAR(TO_DATE('95/10/27', 'RR/MM/DD'), 'YYYY/MM/DD') AS RR형식1,
       TO_CHAR(TO_DATE('95/10/27', 'RR/MM/DD'), 'RRRR/MM/DD') AS RR형식2
FROM   DUAL;

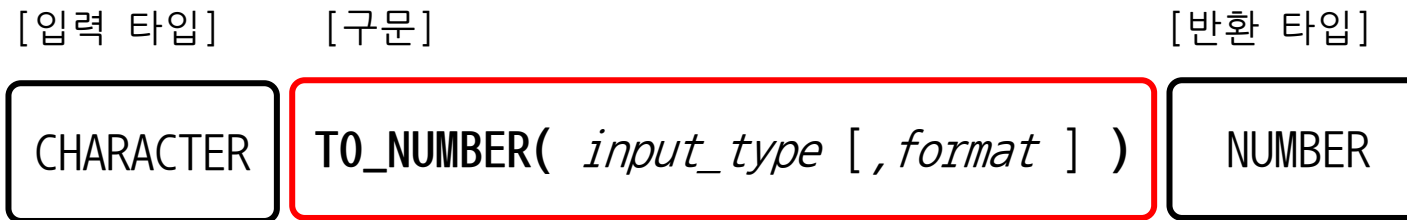
```



현재	입력	YY형식1	YY형식2	RR형식1	RR형식2
2009/10/14	95/10/27	2095/10/27	2095/10/27	1995/10/27	1995/10/27

4.1.3 주요 단일 행 함수 - 데이터 타입 변환 함수 TO_NUMBER

CHARACTER 타입을 NUMBER 타입으로 변환하는 함수



[함수 설명]

- 숫자로 변환될 때 의미 있는 형태의 문자열인 경우
'100' → 100 : 문자열 '100'을 숫자 100으로 변환

4.1.3 주요 단일 행 함수 - 데이터 타입 변환 함수 TO_NUMBER 사용 예

```
SELECT EMP_NAME, EMP_NO,  
       SUBSTR(EMP_NO,1,6)AS 앞부분,  
       SUBSTR(EMP_NO,8) AS 뒷부분,  
       TO_NUMBER( SUBSTR(EMP_NO,1,6) ) + TO_NUMBER( SUBSTR(EMP_NO,8) ) AS 결과  
FROM   EMPLOYEE  
WHERE  EMP_ID = '101';
```



EMP_NAME	EMP_NO	앞부분	뒷부분	결과
강중훈	621136-1006405	621136	1006405	1627541

1006405 + 621136 = 1627541

4.1.3 주요 단일 행 함수 - 기타 함수 NVL

NULL을 지정한 값으로 변환하는 함수

[입력 타입]	[구문]	[반환 타입]
ANY	NVL(<i>expr1</i>, <i>expr2</i>)	ANY

[파라미터]

파라미터	설명
expr1	• NULL을 포함하는 컬럼 • NULL이 없는 경우는 expr1을 반환
expr2	• expr1이 NULL인 경우 변환할 값 • expr1의 데이터 타입과 동일한 타입으로 지정

4.1.3 주요 단일 행 함수 - 기타 함수 NVL 사용 예

```
SELECT EMP_NAME, SALARY, NVL(BONUS_PCT,0)
FROM   EMPLOYEE
WHERE  SALARY > 3500000;
```

EMP_NAME	SALARY	NVL(BONUS_PCT,0)
한선기	9000000	0.2
강중훈	5500000	0
최만식	3600000	0
조재형	3800000	0

```
SELECT EMP_NAME,
       (SALARY*12)+((SALARY*12)*BONUS_PCT)
FROM   EMPLOYEE
WHERE  SALARY > 3500000;
```

EMP_NAME	(SALARY*12)+((SALARY*12)*BONUS_PCT)
한선기	129600000
강중훈	
최만식	
조재형	

NULL에 대한 전체 계산 결과는 NULL임

```
SELECT EMP_NAME,
       (SALARY*12)+
       ((SALARY*12)*NVL(BONUS_PCT,0) )
FROM   EMPLOYEE
WHERE  SALARY > 3500000;
```

EMP_NAME	(SALARY*12)+((SALARY*12)*NVL(BONUS_PCT,0))
한선기	129600000
강중훈	66000000
최만식	43200000
조재형	45600000

4.1.3 주요 단일 행 함수 - 기타 함수 DECODE

SELECT 구문으로 IF-ELSE 논리를 제한적으로 구현한 오라클 DBMS 전용 함수

[입력타입] [구문]

[반환타입]

ANY

`DECODE(expr, search1, result1 [ ,searchN, resultN, ...] [ , default ])`

ANY

[파라미터]

파라미터	설명
<i>expr</i>	대상 컬럼 또는 문자(열)
<i>search</i>	<i>expr</i> 과 비교하려는 값
<i>result</i>	IF <i>expr</i> = <i>search</i> 인 경우의 반환 값
<i>default</i>	• <i>expr</i>과 <i>search</i>가 일치하지 않는 경우의 기본 반환 값 • <i>default</i>를 지정하지 않고 <i>expr</i>과 <i>search</i>가 일치하지 않으면 NULL 반환

4.1.3 주요 단일 행 함수 - 기타 함수 DECODE 사용 예

```
SELECT EMP_NAME,  
       DECODE(SUBSTR(EMP_NO,8,1),  
              '1', '남', '2', '여', '3', '남', '4', '여') AS 성별  
FROM   EMPLOYEE  
WHERE  DEPT_ID = '50';
```

```
SELECT EMP_NAME,  
       DECODE(SUBSTR(EMP_NO,8,1),  
              '1', '남', '3', '남', '여') AS 성별  
FROM   EMPLOYEE  
WHERE  DEPT_ID = '50';
```



EMP_NAME	성별
정지현	여
김예수	여
나승원	남
김순이	여
성해교	여
박하일	남

default 값을 지정하여 코드를 단순화 시킴

4.1.3 주요 단일 행 함수 - 기타 함수 DECODE 사용 예

```
SELECT EMP_ID, EMP_NAME,  
       DECODE(MGR_ID, NULL, '없음', MGR_ID) AS 관리자  
FROM   EMPLOYEE  
WHERE  JOB_ID = 'J4';
```

DECODE 함수는 2개의 NULL을 동일하게 취급함

```
SELECT EMP_ID, EMP_NAME,  
       NVL(MGR_ID, '없음') AS 관리자  
FROM   EMPLOYEE  
WHERE  JOB_ID = 'J4';
```

NULL을 처리하는 경우에는 NVL 함수와 동일한 결과를 나타냄

EMP_ID	EMP_NAME	관리자
103	정도연	104
207	김술오	없음
208	이중기	없음
210	감우섭	없음

4.1.3 주요 단일 행 함수 - 기타 함수 DECODE 사용 예

```

SELECT EMP_NAME,
       JOB_ID,
       SALARY,
       DECODE(JOB_ID,
              'J7', SALARY*1.1,
              'J6', SALARY*1.15,
              'J5', SALARY*1.2,
              SALARY*1.05) AS 인상급여
FROM   EMPLOYEE;

```

EMP_NAME	JOB_ID	SALARY	인상급여
한선기	J1	9000000	9450000
강중훈	J2	5500000	5775000
최만식	J2	3600000	3780000
정도연	J4	2600000	2730000
안석규	J3	3500000	3675000
조재형	J3	3800000	3990000
정지현	J7	1500000	1650000
김예수	J5	2100000	2520000
나승원	J5	2300000	2760000
김순이	J3	3400000	3570000
→ 성해교	J7	1900000	2090000
전우성	J6	2090000	2403500
엄정하	J6	2420000	2783000
심하균		2300000	2415000
고승우	J7	1500000	1650000
박하일	J5	2600000	3120000
권상후	J6	3410000	3921500
임영애	J6	2640000	3036000
엄정하	J7	1500000	1650000
김술오	J4	2500000	2625000
이중기	J4	2500000	2625000
감우섭	J4	2500000	2625000

4.1.3 주요 단일 행 함수 - [참조] CASE

DECODE 함수와 유사한 ANSI 표준 구문

[입력타입] [구문]

[반환타입]

ANY

`CASE expr WHEN search1 THEN result1 [WHEN..THEN..][ELSE default] END`

`CASE WHEN condition1 THEN result1 [WHEN..THEN..][ELSE default] END`

ANY

[파라미터]

파라미터	설명
<code>expr</code>	대상 컬럼 또는 문자(열)
<code>search</code>	<code>expr</code> 과 비교하려는 값
<code>condition</code>	비교 조건
<code>result</code>	- IF <code>expr</code> = <code>search</code> 인 경우의 반환 값 - 비교 조건을 만족시키는 경우의 반환 값
<code>default</code>	<code>expr</code>과 <code>search</code>가 일치하지 않는 경우의 기본 반환 값 - 비교 조건을 만족시키지 않는 경우의 기본 반환 값 <code>default</code>를 지정하지 않으면 일치하지 않거나 조건을 만족시키지 않는 경우 NULL 반환

4.1.3 주요 단일 행 함수 - [참조] CASE 사용 예

```

SELECT EMP_NAME,
       JOB_ID,
       SALARY,
       CASE JOB_ID
         WHEN 'J7' THEN TO_CHAR(SALARY*1.1)
         WHEN 'J6' THEN TO_CHAR(SALARY*1.15)
         WHEN 'J5' THEN TO_CHAR(SALARY*1.2)
         ELSE TO_CHAR(SALARY*1.05) END AS 인상급여
FROM   EMPLOYEE;

```

EMP_NAME	JOB_ID	SALARY	인상급여
한선기	J1	9000000	9450000
강중훈	J2	5500000	5775000
최만식	J2	3600000	3780000
정도연	J4	2600000	2730000
안석규	J3	3500000	3675000
조재형	J3	3800000	3990000
정지현	J7	1500000	1650000
김예수	J5	2100000	2520000
나승원	J5	2300000	2760000
김순이	J3	3400000	3570000
성해교	J7	1900000	2090000
전우성	J6	2090000	2403500
엄정하	J6	2420000	2783000
심하균		2300000	2415000
고승우	J7	1500000	1650000
박하일	J5	2600000	3120000
권상후	J6	3410000	3921500
임영애	J6	2640000	3036000
엄정하	J7	1500000	1650000
김술오	J4	2500000	2625000
이중기	J4	2500000	2625000
감우섭	J4	2500000	2625000

4.1.3 주요 단일 행 함수 - [참조] CASE 사용 예

```

SELECT EMP_ID,
       EMP_NAME,
       SALARY,
       CASE WHEN SALARY <= 3000000 THEN '초급'
            WHEN SALARY <= 4000000 THEN '중급'
            ELSE '고급' END AS 구분
FROM   EMPLOYEE;

```

EMP_ID	EMP_NAME	SALARY	구분
100	한선기	9000000	고급
101	강중훈	5500000	고급
102	최만식	3600000	중급
103	정도연	2600000	초급
104	안석규	3500000	중급
107	조재형	3800000	중급
124	정지현	1500000	초급
141	김예수	2100000	초급
143	나승원	2300000	초급
144	김순이	3400000	중급
149	성해교	1900000	초급
174	전우성	2090000	초급
176	엄정하	2420000	초급
178	심하균	2300000	초급
200	고승우	1500000	초급
201	박하일	2600000	초급
202	권상후	3410000	중급
205	임영애	2640000	초급
206	엄정하	1500000	초급
207	김술오	2500000	초급
208	이중기	2500000	초급
210	감우섭	2500000	초급

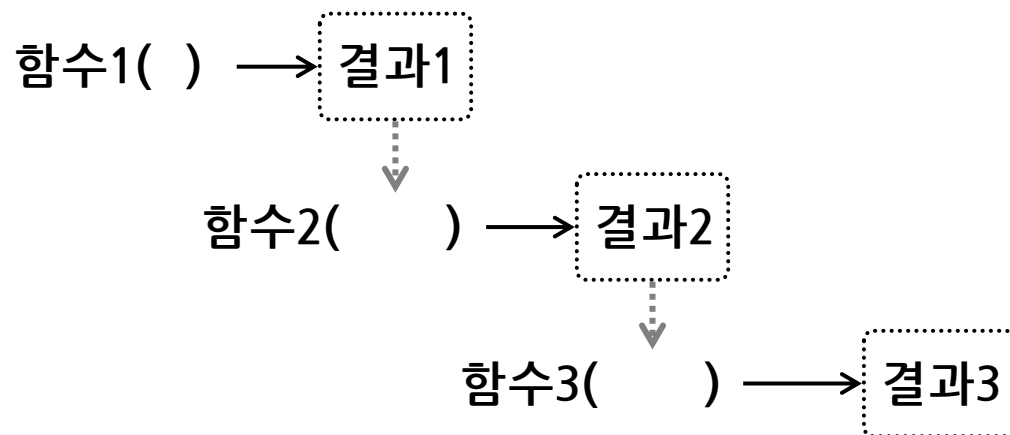
4.1.4 주요 단일 행 함수 - 함수 중첩

- 중첩 사용 가능
- 가장 안쪽의 함수부터 바깥 쪽 방향으로 차례대로 실행
- 먼저 실행된 함수의 반환 값이 바깥 쪽 함수의 입력 값이 됨 → 반환되는 함수 결과 데이터 타입에 주의

[구문]

```
함수3( 함수2( 함수1( ) ) ) )
```

[실행순서]



4.1.4 주요 단일 행 함수 - 함수 중첩

직원 별 Email ID 조회

```

SELECT EMP_NAME,
       EMAIL,
       SUBSTR(EMAIL, 1,
              INSTR(EMAIL, '@')-1) AS ID
FROM   EMPLOYEE;

```

EMP_NAME	EMAIL	ID
한선기	sg_ahn@vcc.com	sg_ahn
강중훈	jh_park@vcc.com	jh_park
최만식	ms_choi@vcc.com	ms_choi
정도연	sy_kang@vcc.com	sy_kang
안석규	sg_han@vcc.com	sg_han
조재형	jh_jo@vcc.com	jh_jo
정지현	jih_jeon@vcc.com	jih_jeon
김예수	hs_kim@vcc.com	hs_kim
나승원	sw_cha@vcc.com	sw_cha
김순이	sm_kim@vcc.com	sm_kim
성해교	hg_song@vcc.com	hg_song
전우성	ws_jeong@vcc.com	ws_jeong
엄정하	jh_um@vcc.com	jh_um
심하균	hk_shin@vcc.com	hk_shin
고승우	sw_jo@vcc.com	sw_jo
박하일	hi_park@vcc.com	hi_park
권상후	sw_kwon@vcc.com	sw_kwon
임영애	jangum_lee@vcc.com	jangum_lee
염정하	jh_yeum@vcc.com	jh_yeum
김술오	so_kim@vcc.com	so_kim
이중기	jk_lee@vcc.com	jk_lee
감우섭	manofking@vcc.com	manofking

4.1.5 주요 그룹 함수

그룹 함수는 NULL을 계산하지 않음

함수	설명
SUM	총합 계산
AVG	• 평균 계산 • NULL을 제외하므로 결과가 달라질 수 있음
MIN	• 최소 값 반환 • DATE 타입: 가장 오래 전 날짜를 의미 • CHARACTER 타입: 해당 character set의 내부 값이 가장 작은 문자를 의미
MAX	• 최대 값 반환 • DATE 타입: 가장 최근 날짜를 의미 • CHARACTER 타입: 해당 character set의 내부 값이 가장 큰 문자를 의미
COUNT	Result Set 전체 행 수 반환

4.1.5 주요 그룹 함수 - SUM

입력 값의 총합을 계산하여 반환하는 함수

[입력 타입]	[구문]	[반환 타입]
NUMBER	SUM([DISTINCT] <i>expr</i>)	NUMBER

```
SELECT SUM(SALARY), SUM(DISTINCT SALARY)
FROM   EMPLOYEE
WHERE  DEPT_ID = '50'
OR     DEPT_ID IS NULL;
```

↓

SUM(SALARY)	SUM(DISTINCT SALARY)
17600000	13800000

- SUM(SALARY): 전체 8건의 합
- SUM(DISTINCT SALARY): 중복 값 제외한 6건의 합

SALARY
1500000
2100000
2300000
3400000
1900000
2300000
2600000
1500000

4.1.5 주요 그룹 함수 - AVG

입력 값의 평균을 계산하여 반환하는 함수



```
SELECT AVG(BONUS_PCT) AS 기본평균,  
       AVG(DISTINCT BONUS_PCT) AS 중복제거평균,  
       AVG(NVL(BONUS_PCT,0)) AS NULL포함평균  
FROM   EMPLOYEE  
WHERE  DEPT_ID IN ('50', '90')  
OR     DEPT_ID IS NULL;
```

BONUS_PCT
0.2
0.1
0.1
0.3

기본평균	중복제거평균	NULL포함평균
0.175	0.2	0.0636363636363636

- 기본평균: BONUS_PCT 값이 있는 4건의 평균
- 중복제거평균: 중복 값을 제외한 3건의 평균
- NULL포함평균: 전체 11건의 평균

4.1.5 주요 그룹 함수 - MIN/MAX

최소 값/최대 값을 반환하는 함수

[입력 타입]

[구문]

[반환 타입]

ANY

MIN(*expr*), MAX(*expr*)

ANY

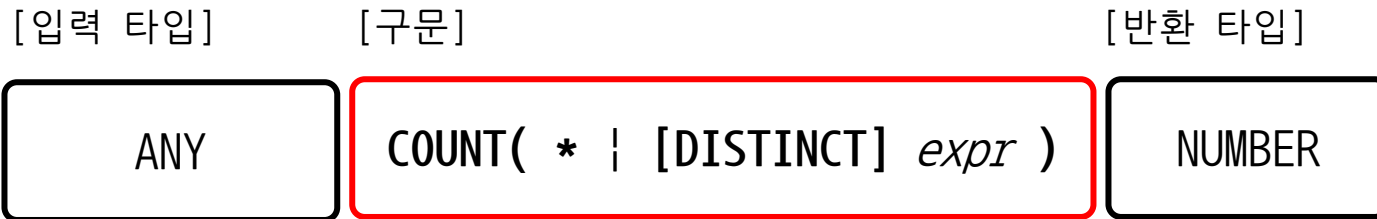
```
SELECT MAX(JOB_ID), MIN(JOB_ID),
       MAX(HIRE_DATE), MIN(HIRE_DATE),
       MAX(SALARY), MIN(SALARY)
FROM   EMPLOYEE
WHERE  DEPT_ID IN ('50','90');
```

JOB_ID	HIRE_DATE	SALARY
J1	90/04/01	9000000
J2	04/04/30	5500000
J2	95/12/30	3600000
J7	04/07/15	1500000
J5	01/03/20	2100000
J5	01/03/20	2300000
J3	99/10/20	3400000
J7	03/08/16	1900000
J5	04/11/10	2600000

MAX(JOB_ID)	MIN(JOB_ID)	MAX(HIRE_DATE)	MIN(HIRE_DATE)	MAX(SALARY)	MIN(SALARY)
J7	J1	90/04/01	95/12/30	9000000	1500000

4.1.5 주요 그룹 함수 - COUNT

Result Set의 행 수를 반환하는 함수



[파라미터]

파라미터	설명
*	Result Set의 전체 행 수 반환
DISTINCT <i>expr</i>	<i>expr</i> 에 포함된 값 중 NULL과 중복 값을 제외한 행 수 반환
<i>expr</i>	<i>expr</i> 에 포함된 값 중 NULL을 제외한 행 수 반환

4.1.5 주요 그룹 함수 - COUNT 사용 예

```
SELECT COUNT(*),  
       COUNT(JOB_ID),  
       COUNT(DISTINCT JOB_ID)  
FROM   EMPLOYEE  
WHERE  DEPT_ID = '50'  
OR     DEPT_ID IS NULL;
```



COUNT(*)	COUNT(JOB_ID)	COUNT(DISTINCT JOB_ID)
8	7	3

JOB_ID
J7
J5
J5
J3
J7
J5
J7

Ch 4.

Additional SELECT

4.1 | 함수 Function

4.2 | **Additional SELECT Option**

- 1) ORDER BY
- 2) GROUP BY
- 3) HAVING
- 4) JOIN
- 5) SET Operator
- 6) Subquery

4.2.1 ORDER BY

SELECT 구문 실행 결과를 특정 컬럼 값 기준으로 정렬할 때 사용

```
SELECT ...  
FROM ...  
WHERE ...  
ORDER BY 기준1 [ ASC | DESC ] [, 기준2 [ASC | DESC], ... ];
```

[구문 설명]

- 항상 SELECT 구문의 맨 마지막에 위치
- 지정한 컬럼 값을 기준으로 정렬
- 정렬 조건
 - ASC : '오름차순'(기본 값)
 - DESC : '내림차순'
- 여러 개의 정렬 기준을 사용하면 기술된 순서대로 적용
- 기준 별로 정렬 조건 구분 적용 가능
- 컬럼 이름, 컬럼 별칭, 컬럼 기술 순서로 표현 가능

4.2.1 ORDER BY - 사용 예

```

SELECT  EMP_NAME, SALARY
FROM    EMPLOYEE
WHERE   DEPT_ID = '50'
OR      DEPT_ID IS NULL
ORDER BY SALARY DESC;

```

EMP_NAME	SALARY
김순이	3400000
박하일	2600000
나승원	2300000
심하균	2300000
김예수	2100000
성해교	1900000
염정하	1500000
정지현	1500000

```

SELECT  EMP_NAME, HIRE_DATE, DEPT_ID
FROM    EMPLOYEE
WHERE   HIRE_DATE > TO_DATE('20030101', 'YYYYMMDD')
ORDER BY DEPT_ID DESC, HIRE_DATE, EMP_NAME;

```

EMP_NAME	HIRE_DATE	DEPT_ID
염정하	03/09/17	
심하균	04/09/30	
강중훈	04/04/30	90
한선기	90/04/01	90
염정하	04/07/21	80
성해교	03/08/16	50
정지현	04/07/15	50
박하일	04/11/10	50
이중기	04/10/01	20
감우섭	05/07/31	20
고승우	03/04/11	10

4.2.1 ORDER BY - 사용 예

```

SELECT  EMP_NAME AS 이름,
        HIRE_DATE AS 입사일,
        DEPT_ID AS 부서코드
FROM    EMPLOYEE
WHERE   HIRE_DATE > TO_DATE('20030101','YYYYMMDD')
ORDER BY 부서코드 DESC, 입사일, 이름;

```

컬럼 별칭을 사용한 경우 별칭 사용 가능

```

SELECT  EMP_NAME AS 이름,
        HIRE_DATE AS 입사일,
        DEPT_ID AS 부서코드
FROM    EMPLOYEE
WHERE   HIRE_DATE > TO_DATE('20030101','YYYYMMDD')
ORDER BY 3 DESC, 2, 1;

```

SELECT 절에 기술된 순서로 표시 가능

이름	입사일	부서코드
염정하	03/09/17	
심하균	04/09/30	
강중훈	04/04/30	90
한선기	90/04/01	90
엄정하	04/07/21	80
성해교	03/08/16	50
정지현	04/07/15	50
박하일	04/11/10	50
이중기	04/10/01	20
감우섭	05/07/31	20
고승우	03/04/11	10

4.2.2 GROUP BY - 하위 데이터 그룹 개념

EMP_NAME	SALARY	DEPT_ID
권상후	3410000	10
임영애	2640000	10
고승우	1500000	10
김술오	2500000	20
이중기	2500000	20
감우섭	2500000	20
김순이	3400000	50
나승원	2300000	50
김예수	2100000	50
정지현	1500000	50
성해교	1900000	50
박하일	2600000	50
정도연	2600000	60
조재형	3800000	60
안석규	3500000	60
전우성	2090000	80
엄정하	2420000	80
한선기	9000000	90
최만식	3600000	90
강중훈	5500000	90
염정하	1500000	
심하균	2300000	

EMP_NAME	SALARY	DEPT_ID
권상후	3410000	10
임영애	2640000	10
고승우	1500000	10
김술오	2500000	20
이중기	2500000	20
감우섭	2500000	20
김순이	3400000	50
나승원	2300000	50
김예수	2100000	50
정지현	1500000	50
성해교	1900000	50
박하일	2600000	50
정도연	2600000	60
조재형	3800000	60
안석규	3500000	60
전우성	2090000	80
엄정하	2420000	80
한선기	9000000	90
최만식	3600000	90
강중훈	5500000	90
염정하	1500000	
심하균	2300000	

하위그룹1

하위그룹2

하위그룹3

하위그룹4

하위그룹5

하위그룹6

하위그룹7

DEPT_ID	SUM(SALARY)
	3800000
50	13800000
20	7500000
10	7550000
90	18100000
80	4510000
60	9900000

- 각 부서가 하나의 데이터 그룹
- 부서 수 만큼 데이터 그룹 생성
- 그룹 별(부서 별)로 그룹함수 적용

4.2.2 GROUP BY

```
SELECT ...  
FROM ...  
WHERE ...  
GROUP BY column_name | expr  
ORDER BY 기준1 [ ASC | DESC ] [, 기준2 [ ASC | DESC ], ... ];
```

[구문 설명]

- GROUP BY 절에 기술한 컬럼이나 표현식을 기준으로 데이터 그룹 생성
- 각 그룹 별로 SELECT 절에 기술한 그룹 함수가 적용
- SELECT 절에 기술한 컬럼 중, 그룹 함수에 사용되지 않은 컬럼은 GROUP BY 절에 반드시 기술되어야 함
- 제약 사항
 - WHERE 절에는 그룹 함수를 사용할 수 없음
 - GROUP BY 절에는 컬럼 이름만 사용 가능(별칭, 순서 사용 불가)

4.2.2 GROUP BY

```
SELECT  DEPT_ID AS 부서,  
        ROUND(AVG(SALARY),-4) AS 평균급여  
FROM    EMPLOYEE  
GROUP BY DEPT_ID  
ORDER BY 1;
```

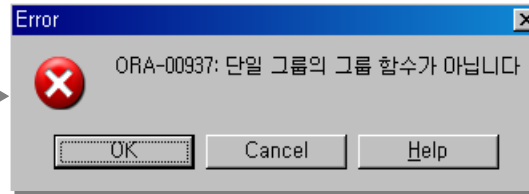
부서	평균급여
10	2520000
20	2500000
50	2300000
60	3300000
80	2260000
90	6030000
	1900000

```
SELECT  DECODE(SUBSTR(EMP_NO,8,1),  
              '1', '남', '3', '남', '여') AS 성별,  
        ROUND(AVG(SALARY),-4) AS 평균급여  
FROM    EMPLOYEE  
GROUP BY DECODE(SUBSTR(EMP_NO,8,1),  
              '1', '남', '3', '남', '여')  
ORDER BY 2;
```

성별	평균급여
여	2260000
남	3360000

4.2.2 GROUP BY 사용 예

```
SELECT DEPT_ID, COUNT(*)  
FROM   EMPLOYEE;
```



'ORA-00937' 에러가 발생하면
GROUP BY 가 누락되었는지
확인

그룹 함수 COUNT는 유일한 반환 값만 생성할 수 있는데, SELECT 절에서는 여러 개의 DEPT_ID 값을 함께 표시하도록 작성하였음 → 그룹 함수가 동작해야 하는 그룹을 찾지 못한 경우

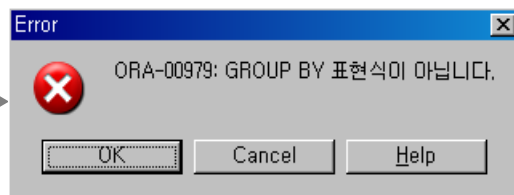
[수정 구문]

```
SELECT  DEPT_ID,  
        COUNT(*)  
FROM    EMPLOYEE  
GROUP BY DEPT_ID  
ORDER BY 1;
```

DEPT_ID	COUNT(*)
10	3
20	3
50	6
60	3
80	2
90	3
	2

4.2.2 GROUP BY 사용 예

```
SELECT EMP_NAME, DEPT_ID, COUNT(*)
FROM   EMPLOYEE
GROUP BY DEPT_ID;
```



'ORA-00979' 에러가 발생하면
GROUP BY 절에 누락된 select
list가 있는지 확인

[수정 구문]

```
SELECT   EMP_NAME, DEPT_ID, COUNT(*)
FROM     EMPLOYEE
GROUP BY EMP_NAME, DEPT_ID;
```



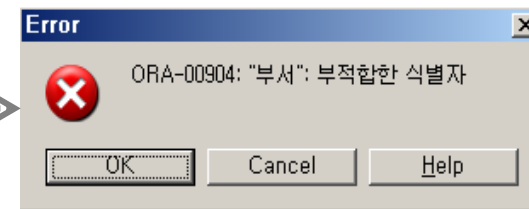
결과(일부)

EMP_NAME	DEPT_ID	COUNT(*)
조재형	60	1
성해교	50	1
심하균		1
김예수	50	1
박하일	50	1
임영애	10	1
염정하		1
이중기	20	1
전우성	80	1
권상후	10	1

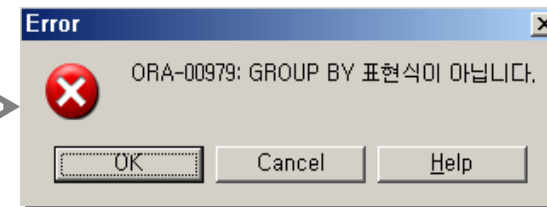
4.2.2 GROUP BY 사용 예

컬럼 별칭이나 컬럼 기술 순서는 사용할 수 없음

```
SELECT  DEPT_ID AS 부서,  
        SUM(SALARY)  
FROM    EMPLOYEE  
GROUP BY 부서;
```



```
SELECT  DEPT_ID AS 부서,  
        SUM(SALARY)  
FROM    EMPLOYEE  
GROUP BY 1;
```



4.2.2 GROUP BY 사용 예

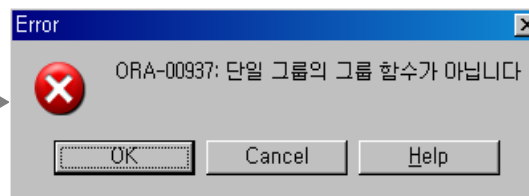
```
SELECT  MAX(SUM(SALARY))  
FROM    EMPLOYEE  
GROUP BY DEPT_ID;
```

MAX(SUM(SALARY))
181000000

그룹 함수는 2번까지
중첩 사용 가능

DEPT_ID	SUM(SALARY)
	38000000
50	138000000
20	75000000
10	75500000
90	181000000
80	45100000
60	99000000

```
SELECT  DEPT_ID,  
        MAX(SUM(SALARY))  
FROM    EMPLOYEE  
GROUP BY DEPT_ID;
```



4.2.3 HAVING

GROUP BY에 의해 그룹화 된 데이터에 대한 그룹 함수 실행 결과를 제한하기 위해 사용
(WHERE는 테이블에 포함된 원본 데이터를 제한하기 위해 사용)

```
SELECT ...  
FROM ...  
WHERE ...  
GROUP BY column_name | expr  
HAVING condition  
ORDER BY 기준1 [ ASC | DESC] [, 기준2 [ASC | DESC], ... ];
```

4.2.3 HAVING

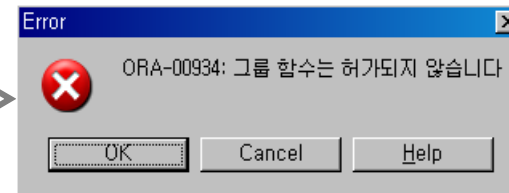
```
SELECT  DEPT_ID, SUM(SALARY)
FROM    EMPLOYEE
GROUP BY DEPT_ID
HAVING  SUM(SALARY) > 9000000;
```



DEPT_ID	SUM(SALARY)
50	13800000
90	18100000
60	9900000

부서 별 급여 총합을 계산한 결과 중 9000000 이상인 경우만 선택

```
SELECT  DEPT_ID, SUM(SALARY)
FROM    EMPLOYEE
WHERE    SUM(SALARY) > 9000000
GROUP BY DEPT_ID;
```



WHERE 절에는 그룹 함수를 사용할 수 없음

→ WHERE 절이 수행되어야 그룹 함수가 실행 될 대상 그룹이 결정

4.2.4 JOIN - 필요성

'EMP_NAME' 컬럼은 EMPLOYEE 테이블에 있고, 'DEPT_NAME' 컬럼은 DEPARTMENT 테이블에 있을 때 두 컬럼을 한번에 볼 수 있는 방법이 필요함

	EMP_NAME	DEPT_ID
EMPLOYEE	한선기	90
	강중훈	90
	최만식	90
	정도연	60
	안석규	60
	조재형	60
	정지현	50
	김예수	50
	나승원	50
	김순이	50
	성해교	50
	전우성	80
	엄정하	80
	심하균	
	고승우	10
	박하일	50
	권상후	10
	임영애	10
	염정하	
	김술오	20
	이중기	20
	감우섭	20

DEPT_ID	DEPT_NAME
20	회계팀
10	본사 인사팀
50	해외영업1팀
60	기술지원팀
80	해외영업2팀
90	해외영업3팀
30	마케팅팀

DEPARTMENT

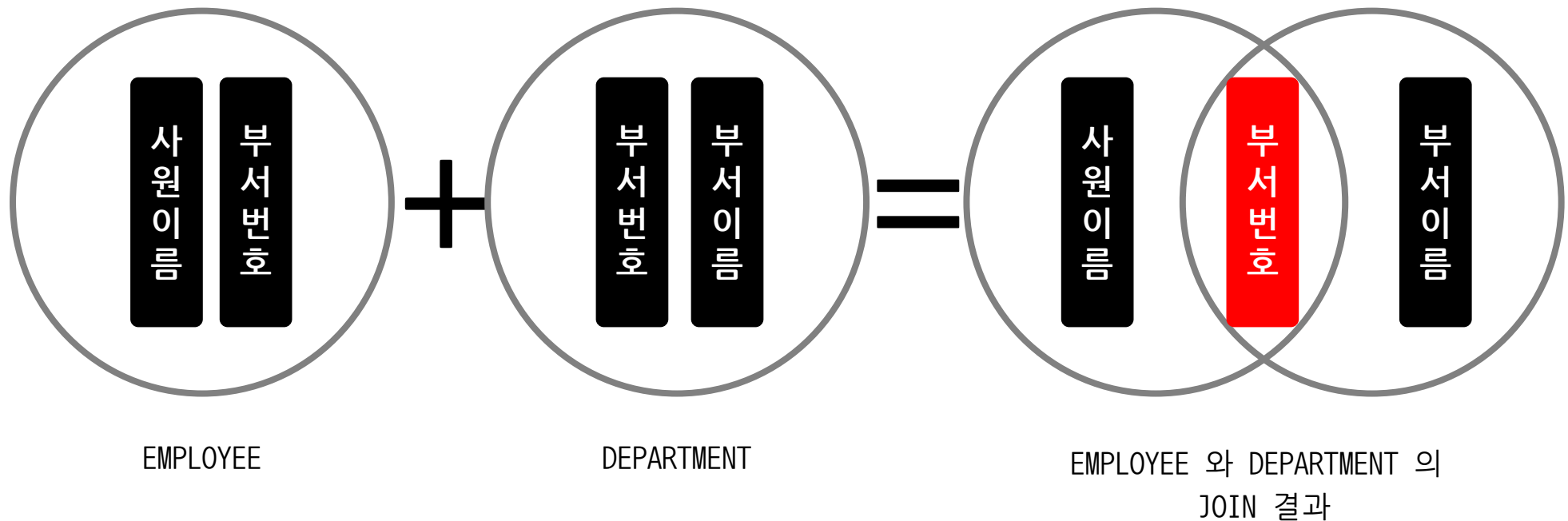
EMP_NAME	DEPT_NAME
감우섭	회계팀
이중기	회계팀
김술오	회계팀
임영애	본사 인사팀
권상후	본사 인사팀
고승우	본사 인사팀
박하일	해외영업1팀
성해교	해외영업1팀
김순이	해외영업1팀
나승원	해외영업1팀
김예수	해외영업1팀
정지현	해외영업1팀
조재형	기술지원팀
안석규	기술지원팀
정도연	기술지원팀
엄정하	해외영업2팀
전우성	해외영업2팀
최만식	해외영업3팀
강중훈	해외영업3팀
한선기	해외영업3팀
염정하	
심하균	

????

4.2.4 JOIN - 개념

서로 연관되고 다른 테이블에 존재하는 컬럼들을 한번에 조회하기 위해 사용하는 대표적인 기법

[JOIN 개념 도식 1]



4.2.4 JOIN - 오라클 전용 구문

```

SELECT EMP_NAME, DEPT_NAME
FROM   EMPLOYEE E,
       DEPARTMENT D
WHERE  E.DEPT_ID = D.DEPT_ID;

```

- FROM 절에 조회 대상 테이블을 심표로 구분하여 기술
- WHERE 절에 테이블 사이의 관계를 표시하는 조건 기술
- 동일한 이름의 컬럼이 여러 테이블에 존재하는 경우
→ SELECT 절/WHERE 절에 컬럼 이름을 기술할 때 어떤
테이블에 포함된 컬럼인지 구분해서 표시
- 테이블 이름도 별칭을 사용할 수 있다

EMPLOYEE(일부)

EMP_NAME	DEPT_ID
한선기	90
강중훈	90
최만식	90
정도연	60
안석규	60
조재형	60
정지현	50
김예수	50
나승원	50
김순이	50
성해교	50

DEPARTMENT

DEPT_ID	DEPT_NAME
20	회계팀
10	본사 인사팀
50	해외영업1팀
60	기술지원팀
80	해외영업2팀
90	해외영업3팀
30	마케팅팀

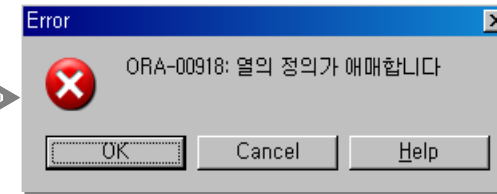
????(일부)

EMP_NAME	DEPT_NAME
감우섭	회계팀
이중기	회계팀
김술오	회계팀
임영애	본사 인사팀
권상후	본사 인사팀
고승우	본사 인사팀
박하일	해외영업1팀
성해교	해외영업1팀
김순이	해외영업1팀

4.2.4 JOIN - 오라클 전용 구문

```
SELECT EMP_NAME, DEPT_NAME  
FROM   EMPLOYEE, DEPARTMENT  
WHERE  DEPT_ID = DEPT_ID;
```

```
SELECT EMP_NAME, DEPT_ID,  
       DEPT_NAME  
FROM   EMPLOYEE E,  
       DEPARTMENT D  
WHERE  E.DEPT_ID = D.DEPT_ID;
```



양쪽 테이블에 동일한 이름의 컬럼이 모두 존재하는 경우

- WHERE 절: 테이블 구분 필요(테이블 별칭 사용 가능)
- SELECT 절: 양쪽 테이블의 컬럼 값은 동일하지만, 문법 상 어떤 테이블의 컬럼 값을 표시할 것인지 구분 필요

4.2.4 JOIN - ANSI 표준 구문

- JOIN 유형을 세분화
- WHERE 절에서 JOIN 조건을 별도로 분리하고 'JOIN' 키워드를 명시적으로 사용

```
SELECT ...  
FROM   table1  
{[INNER] JOIN table2 ON (condition1 [AND condition2 ...]) |  
  [INNER] JOIN table2 USING (column1 [, ...]) |  
  NATURAL [INNER] JOIN table2 |  
  LEFT|RIGHT|FULL [OUTER] JOIN table2 ON (condition1 [AND condition2 ...]) |  
  LEFT|RIGHT|FULL [OUTER] JOIN table2 USING (column1 [, ...]) |  
  CROSS JOIN table2 }  
WHERE  ...  
GROUP BY column_name | expr  
HAVING condition  
ORDER BY 기준1 [ ASC | DESC ] [, 기준2 [ASC | DESC], ... ];
```

4.2.4 JOIN - ANSI 표준 구문 ¹⁾JOIN USING

조인 조건으로 사용하는 컬럼 이름이 동일한 경우 사용

```
SELECT EMP_NAME, DEPT_NAME
FROM   EMPLOYEE
JOIN   DEPARTMENT USING (DEPT_ID)
WHERE  JOB_ID = 'J6';
```

- 조인 조건에 사용되는 컬럼은 SELECT 절이나 JOIN 절에서 테이블 구분이 필요 없음
- 조인 조건으로 컬럼 여러 개를 사용하려면 쉼표로 구분
- 테이블 별칭은 사용할 수 없음



EMP_NAME	DEPT_NAME
전우성	해외영업2팀
엄정하	해외영업2팀
권상후	본사 인사팀
임영애	본사 인사팀

4.2.4 JOIN - ANSI 표준 구문 ¹⁾JOIN USING

```
SELECT EMP_NAME, LOC_ID  
FROM   EMPLOYEE2  
JOIN   DEPARTMENT USING (DEPT_ID, LOC_ID);
```

- EMPLOYEE2 테이블은 EMPLOYEE 테이블에 LOC_ID 컬럼이 추가된 구조
- 모두 'A1' 이 입력되어 있음



EMP_NAME	LOC_ID
고승우	A1
권상후	A1
임영애	A1
김술오	A1
이중기	A1
감우섭	A1

4.2.4 JOIN - ANSI 표준 구문 ²⁾JOIN ON

조인 조건으로 사용하는 컬럼 이름이 서로 다른 경우 사용

```
SELECT DEPT_NAME,  
       LOC_DESCRIBE  
FROM   DEPARTMENT  
JOIN   LOCATION ON (LOC_ID = LOCATION_ID);
```



DEPT_NAME	LOC_DESCRIBE
회계팀	아시아지역1
본사 인사팀	아시아지역1
해외영업1팀	미주지역
기술지원팀	기타지역
해외영업2팀	아시아지역2
해외영업3팀	아시아지역3
마케팅팀	아시아지역1

4.2.4 JOIN - OUTER JOIN

조건을 만족시키지 못하는 행까지 Result Set에 포함시키는 조인 유형

EMP_NAME	DEPT_ID
한선기	90
강중훈	90
최만식	90
정도연	60
안석규	60
조재형	60
정지현	50
김예수	50
나승원	50
김순이	50
성해교	50
전우성	80
엄정하	80
심하균	
고승우	10
박하일	50
권상후	10
임영애	10
염정하	
김술오	20
이중기	20
감우섭	20

DEPT_ID	DEPT_NAME
20	회계팀
10	본사 인사팀
50	해외영업1팀
60	기술지원팀
80	해외영업2팀
90	해외영업3팀
30	마케팅팀

EMP_NAME	DEPT_NAME
한선기	해외영업3팀
강중훈	해외영업3팀
최만식	해외영업3팀
정도연	기술지원팀
안석규	기술지원팀
조재형	기술지원팀
정지현	해외영업1팀
김예수	해외영업1팀
나승원	해외영업1팀
김순이	해외영업1팀
성해교	해외영업1팀
전우성	해외영업2팀
엄정하	해외영업2팀
고승우	본사 인사팀
박하일	해외영업1팀
권상후	본사 인사팀
임영애	본사 인사팀
김술오	회계팀
이중기	회계팀
감우섭	회계팀

EMP_NAME	DEPT_NAME
감우섭	회계팀
이중기	회계팀
김술오	회계팀
임영애	본사 인사팀
권상후	본사 인사팀
고승우	본사 인사팀
박하일	해외영업1팀
성해교	해외영업1팀
김순이	해외영업1팀
나승원	해외영업1팀
김예수	해외영업1팀
정지현	해외영업1팀
조재형	기술지원팀
안석규	기술지원팀
정도연	기술지원팀
엄정하	해외영업2팀
전우성	해외영업2팀
최만식	해외영업3팀
강중훈	해외영업3팀
한선기	해외영업3팀
염정하	
심하균	

4.2.4 JOIN - OUTER JOIN ¹⁾오라클 전용 구문

- 연산자 '+' 사용
- 조인 조건을 만족시키는 행이 없는 테이블 기준

```
SELECT EMP_NAME, DEPT_NAME
FROM   EMPLOYEE E, DEPARTMENT D
WHERE  E.DEPT_ID = D.DEPT_ID(+);
```

소속 부서가 없는 직원까지 포함하는 의미

EMP_NAME	DEPT_NAME
감우섭	회계팀
이중기	회계팀
김술오	회계팀
임영애	본사 인사팀
권상후	본사 인사팀
고승우	본사 인사팀
박하일	해외영업1팀
성해교	해외영업1팀
김순이	해외영업1팀
나승원	해외영업1팀
김예수	해외영업1팀
정지현	해외영업1팀
조재형	기술지원팀
안석규	기술지원팀
정도연	기술지원팀
엄정하	해외영업2팀
전우성	해외영업2팀
최만식	해외영업3팀
강중훈	해외영업3팀
한선기	해외영업3팀
염정하	
심하균	

EMP_NAME	DEPT_ID
한선기	90
강중훈	90
최만식	90
정도연	60
안석규	60
조재형	60
정지현	50
김예수	50
나승원	50
김순이	50
성해교	50
전우성	80
엄정하	80
심하균	
고승우	10
박하일	50
권상후	10
임영애	10
염정하	
김술오	20
이중기	20
감우섭	20

DEPT_ID	DEPT_NAME
20	회계팀
10	본사 인사팀
50	해외영업1팀
60	기술지원팀
80	해외영업2팀
90	해외영업3팀
30	마케팅팀

--	--

4.2.4 JOIN - OUTER JOIN ¹⁾오라클 전용 구문

```
SELECT EMP_NAME, DEPT_NAME
FROM   EMPLOYEE E, DEPARTMENT D
WHERE  D.DEPT_ID = E.DEPT_ID(+);
```

소속 직원이 없는 부서까지 포함하는 의미

EMP_NAME	DEPT_NAME
한선기	해외영업3팀
강중훈	해외영업3팀
최만식	해외영업3팀
정도연	기술지원팀
안석규	기술지원팀
조재형	기술지원팀
정지현	해외영업1팀
김예수	해외영업1팀
나승원	해외영업1팀
김순이	해외영업1팀
성해교	해외영업1팀
전우성	해외영업2팀
엄정하	해외영업2팀
고승우	본사 인사팀
박하일	해외영업1팀
권상후	본사 인사팀
임영애	본사 인사팀
김술오	회계팀
이중기	회계팀
감우섭	회계팀
	마케팅팀

EMP_NAME	DEPT_ID
한선기	90
강중훈	90
최만식	90
정도연	60
안석규	60
조재형	60
정지현	50
김예수	50
나승원	50
김순이	50
성해교	50
전우성	80
엄정하	80
심하균	
고승우	10
박하일	50
권상후	10
임영애	10
염정하	
김술오	20
이중기	20
감우섭	20
	30

DEPT_ID	DEPT_NAME
20	회계팀
10	본사 인사팀
50	해외영업1팀
60	기술지원팀
80	해외영업2팀
90	해외영업3팀
30	마케팅팀

4.2.4 JOIN - OUTER JOIN ²⁾ANSI 표준 구문

- 전체 행을 모두 포함시켜야 하는 테이블 기준
- LEFT, RIGHT 키워드 사용
 - LEFT: 기준 테이블이 JOIN 키워드보다 먼저 기술된 경우
 - RIGHT: 기준 테이블이 JOIN 키워드보다 나중에 기술된 경우

```
SELECT EMP_NAME, DEPT_NAME
FROM   EMPLOYEE
LEFT   JOIN DEPARTMENT USING (DEPT_ID)
ORDER BY 1;
```

```
SELECT EMP_NAME, DEPT_NAME
FROM   DEPARTMENT
RIGHT  JOIN EMPLOYEE USING (DEPT_ID)
ORDER BY 1;
```

EMP_NAME	DEPT_NAME
감우섭	회계팀
이중기	회계팀
김술오	회계팀
임영애	본사 인사팀
권상후	본사 인사팀
고승우	본사 인사팀
박하일	해외영업1팀
성해교	해외영업1팀
김순이	해외영업1팀
나승원	해외영업1팀
김예수	해외영업1팀
정지현	해외영업1팀
조재형	기술지원팀
안석규	기술지원팀
정도연	기술지원팀
엄정하	해외영업2팀
전우성	해외영업2팀
최만식	해외영업3팀
강중훈	해외영업3팀
한선기	해외영업3팀
염정하	
심하균	

4.2.4 JOIN - OUTER JOIN ²⁾ANSI 표준 구문

```
SELECT EMP_NAME, DEPT_NAME
FROM   DEPARTMENT
LEFT   JOIN EMPLOYEE USING (DEPT_ID);
```

```
SELECT EMP_NAME, DEPT_NAME
FROM   EMPLOYEE
RIGHT  JOIN DEPARTMENT USING (DEPT_ID);
```

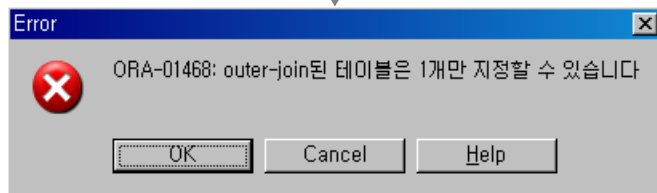
EMP_NAME	DEPT_NAME
한선기	해외영업3팀
강중훈	해외영업3팀
최만식	해외영업3팀
정도연	기술지원팀
안석규	기술지원팀
조재형	기술지원팀
정지현	해외영업1팀
김예수	해외영업1팀
나승원	해외영업1팀
김순이	해외영업1팀
성해교	해외영업1팀
전우성	해외영업2팀
엄정하	해외영업2팀
고승우	본사 인사팀
박하일	해외영업1팀
권상후	본사 인사팀
임영애	본사 인사팀
김술오	회계팀
이중기	회계팀
감우섭	회계팀
	마케팅팀

4.2.4 JOIN - FULL OUTER JOIN

- 양쪽 테이블을 동시에 OUTER JOIN하는 ANSI 표준 구문
- 오라클 전용 구문은 지원되지 않음

```
SELECT EMP_NAME, DEPT_NAME
FROM   EMPLOYEE
FULL   JOIN DEPARTMENT USING (DEPT_ID);
```

```
SELECT EMP_NAME, DEPT_NAME
FROM   EMPLOYEE E, DEPARTMENT D
WHERE  E.DEPT_ID(+) = D.DEPT_ID(+);
```



EMP_NAME	DEPT_NAME
감우섭	회계팀
이중기	회계팀
김솔오	회계팀
임영애	본사 인사팀
권상후	본사 인사팀
고승우	본사 인사팀
박하일	해외영업1팀
성해교	해외영업1팀
김순이	해외영업1팀
나승원	해외영업1팀
김예수	해외영업1팀
정지현	해외영업1팀
조재형	기술지원팀
안석규	기술지원팀
정도연	기술지원팀
엄정하	해외영업2팀
전우성	해외영업2팀
최만식	해외영업3팀
강중훈	해외영업3팀
한선기	해외영업3팀
엄정하	
심하균	
	마케팅팀

4.2.4 JOIN - CROSS JOIN

- 대상 테이블의 모든 행에 대해 가능한 모든 조합을 생성하는 ANSI 표준 구문
- 오라클 구문에서는 조인 조건이 누락된 경우를 의미

결과(일부)

```
SELECT *
FROM   COUNTRY
CROSS JOIN LOCATION;
```

```
SELECT *
FROM   COUNTRY, LOCATION;
```

COUNTRY_ID	COUNTRY_NAME	LOCATION_ID	COUNTRY_ID	LOC_DESCRIBE
KO	한국	A1	KO	아시아지역1
JP	일본	A1	KO	아시아지역1
CH	중국	A1	KO	아시아지역1
US	미국	A1	KO	아시아지역1
ID	인도	A1	KO	아시아지역1
KO	한국	A2	JP	아시아지역2
JP	일본	A2	JP	아시아지역2
CH	중국	A2	JP	아시아지역2

[COUNTRY]

COUNTRY_ID	COUNTRY_NAME
KO	한국
JP	일본
CH	중국
US	미국
ID	인도

[LOCATION]

LOCATION_ID	COUNTRY_ID	LOC_DESCRIBE
A1	KO	아시아지역1
A2	JP	아시아지역2
A3	CH	아시아지역3
U1	US	미주지역
OT	ID	기타지역

COUNTRY 테이블의 5건과 LOCATION 테이블의 5건에 대한 모든 조합 25건이 생성

4.2.4 JOIN - Non Equijoin

```

SELECT EMP_NAME, SALARY, SLEVEL
FROM   EMPLOYEE
JOIN   SAL_GRADE ON (SALARY BETWEEN LOWEST AND HIGHEST)
ORDER BY 3;

```

[SAL_GRADE]

SLEVEL	LOWEST	HIGHEST
A	3000000	9000000
B	2500000	2999999
C	2000000	2499999
D	1500000	1999999
E	1000000	1499999

컬럼 값이 같은 경우가 아닌
범위에 속하는지의 여부를
확인하는 의미

결과(일부)

EMP_NAME	SALARY	SLEVEL
강중훈	5500000	A
조재형	3800000	A
최만식	3600000	A
안석규	3500000	A
권상후	3410000	A
김순이	3400000	A
한선기	9000000	A
감우섭	2500000	B
임영애	2640000	B
박하일	2600000	B

4.2.4 JOIN - SELF JOIN

- 한 테이블을 두 번 조인하는 유형
- 테이블 별칭을 사용해야 함

```
SELECT E.EMP_NAME AS 직원,
       M.EMP_NAME AS 관리자
FROM   EMPLOYEE E
JOIN   EMPLOYEE M ON (E.MGR_ID = M.EMP_ID)
ORDER BY 1;
```



직원	관리자
강중훈	한선기
고승우	한선기
권상후	고승우
김순이	김예수
김예수	한선기
나승원	김예수
성해교	김예수
안석규	한선기
엄정하	전우성
임영애	고승우
전우성	한선기
정도연	안석규
정지현	김예수
조재형	안석규
최만식	강중훈

4.2.4 JOIN - N개 테이블 JOIN

N개의 테이블을 조인하려면 최소 N-1개의 조인 조건(또는 N-1개의 JOIN 키워드)이 필요

[EMPLOYEE]			[JOB]		[EMPLOYEE]			[DEPARTMENT]	
EMP_NAME	JOB_TITLE	DEPT_NAME	JOB_ID	JOB_TITLE	EMP_NAME	JOB_ID	DEPT_ID	DEPT_ID	DEPT_NAME
한선기	대표이사	해외영업3팀	J1	대표이사	한선기	J1	90	20	회계팀
최만식	부사장	해외영업3팀	J2	부사장	강중훈	J2	90	10	본사 인사팀
강중훈	부사장	해외영업3팀	J3	부장	최만식	J2	90	50	해외영업1팀
김순이	부장	해외영업1팀	J4	차장	정도연	J4	60	60	기술지원팀
조재형	부장	기술지원팀	J5	과장	안석규	J3	60	80	해외영업2팀
안석규	부장	기술지원팀	J6	대리	조재형	J3	60	90	해외영업3팀
감우섭	차장	회계팀	J7	사원	정지현	J7	50	30	마케팅팀
이중기	차장	회계팀			김예수	J5	50		
김술오	차장	회계팀			나승원	J5	50		
정도연	차장	기술지원팀			김순이	J3	50		
박하일	과장	해외영업1팀			성해교	J7	50		
나승원	과장	해외영업1팀			전우성	J6	80		
김예수	과장	해외영업1팀			엄정하	J6	80		
임영애	대리	본사 인사팀			심하균				
권상후	대리	본사 인사팀			고승우	J7	10		
엄정하	대리	해외영업2팀			박하일	J5	50		
전우성	대리	해외영업2팀			권상후	J6	10		
고승우	사원	본사 인사팀			임영애	J6	10		
성해교	사원	해외영업1팀			염정하	J7			
정지현	사원	해외영업1팀			김술오	J4	20		
					이중기	J4	20		
					감우섭	J4	20		

4.2.4 JOIN - N개 테이블 JOIN

```

SELECT EMP_NAME,
       JOB_TITLE,
       DEPT_NAME
FROM   EMPLOYEE
JOIN   JOB USING (JOB_ID)
JOIN   DEPARTMENT USING (DEPT_ID);

```

- EMPLOYEE와 JOB이 먼저 조인되어 새로운 테이블처럼 취급
- 그 결과와 DEPARTMENT가 조인되는 개념

EMP_NAME	JOB_TITLE	DEPT_ID	DEPT_ID	DEPT_NAME
한선기	대표이사	90	20	회계팀
강중훈	부사장	90	10	본사 인사팀
최만식	부사장	90	50	해외영업1팀
정도연	차장	60	60	기술지원팀
안석규	부장	60	80	해외영업2팀
조재형	부장	60	90	해외영업3팀
정지현	사원	50	30	마케팅팀
김예수	과장	50		
나승원	과장	50		
김순이	부장	50		
성해교	사원	50		
전우성	대리	80		
엄정하	대리	80		
고승우	사원	10		
박하일	과장	50		
권상후	대리	10		
임영애	대리	10		
염정하	사원			
김술오	차장	20		
이중기	차장	20		
감우섭	차장	20		

4.2.4 JOIN - N개 테이블 JOIN

```

SELECT EMP_NAME,
       JOB_TITLE,
       DEPT_NAME
FROM   EMPLOYEE
LEFT JOIN JOB USING (JOB_ID)
LEFT JOIN DEPARTMENT USING (DEPT_ID);

```

EMPLOYEE 테이블의 JOB_ID 컬럼과 DEPT_ID 컬럼이 조인 조건으로 사용되고 있기 때문에 해당 정보가 없는 직원을 결과에 포함하려면 OUTER JOIN을 사용해야 함

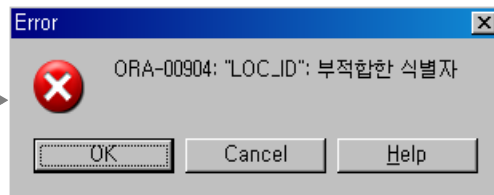
EMP_NAME	JOB_TITLE	DEPT_NAME
김술오	차장	회계팀
이중기	차장	회계팀
감우섭	차장	회계팀
고승우	사원	본사 인사팀
권상후	대리	본사 인사팀
임영애	대리	본사 인사팀
정지현	사원	해외영업1팀
성해교	사원	해외영업1팀
김예수	과장	해외영업1팀
나승원	과장	해외영업1팀
박하일	과장	해외영업1팀
김순이	부장	해외영업1팀
정도연	차장	기술지원팀
안석규	부장	기술지원팀
조재형	부장	기술지원팀
전우성	대리	해외영업2팀
엄정하	대리	해외영업2팀
강중훈	부사장	해외영업3팀
최만식	부사장	해외영업3팀
한선기	대표이사	해외영업3팀
심하균		
엄정하	사원	

4.2.4 JOIN - N개 테이블 JOIN

```

SELECT EMP_NAME,
       LOC_DESCRIBE,
       DEPT_NAME
FROM   EMPLOYEE
JOIN   LOCATION ON (LOCATION_ID = LOC_ID)
JOIN   DEPARTMENT USING (DEPT_ID);

```



- 구문 순서대로 EMPLOYEE와 LOCATION이 먼저 조인 되어야 함
- 두 테이블은 직접적인 연관 관계가 없으므로 오류 발생

```

SELECT EMP_NAME,
       LOC_DESCRIBE,
       DEPT_NAME
FROM   EMPLOYEE
JOIN   DEPARTMENT USING (DEPT_ID)
JOIN   LOCATION ON (LOCATION_ID = LOC_ID);

```

결과(일부)

EMP_NAME	LOC_DESCRIBE	DEPT_NAME
한선기	아시아지역3	해외영업3팀
강중훈	아시아지역3	해외영업3팀
최만식	아시아지역3	해외영업3팀
정도연	기타지역	기술지원팀
안석규	기타지역	기술지원팀
조재형	기타지역	기술지원팀

DEPARTMENT 테이블이 EMPLOYEE와 LOCATION을 연결시킬 수 있도록 구문을 수정해야 함

4.2.4 JOIN - N개 테이블 JOIN

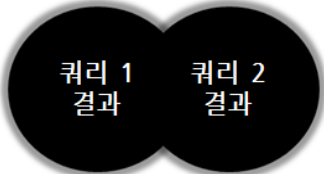
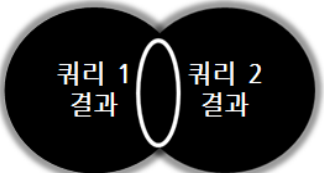
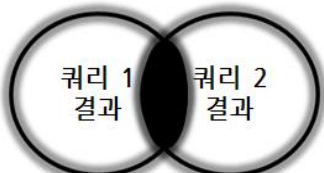
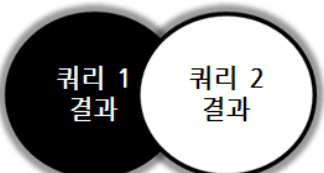
```
SELECT EMP_NAME,  
       DEPT_NAME  
FROM   EMPLOYEE  
JOIN   JOB USING (JOB_ID)  
JOIN   DEPARTMENT USING (DEPT_ID)  
JOIN   LOCATION ON (LOC_ID = LOCATION_ID)  
WHERE  JOB_TITLE='대리'  
AND    LOC_DESCRIBE LIKE '아시아%';
```

EMP_NAME	DEPT_NAME
전우성	해외영업2팀
엄정하	해외영업2팀
권상후	본사 인사팀
임영애	본사 인사팀

- 컬럼 조회 목적이 아닌 WHERE 조건을 위해 조인에 포함되어야 하는 테이블이 필요
- JOB 테이블은 직급 조건을 위해, LOCATION 테이블은 지역 조건을 위해 조인되었음

4.2.5 SET Operator - 개념

- 두 개 이상의 쿼리 결과를 하나로 결합시키는 연산자
- SELECT 절에 기술하는 컬럼 개수와 데이터 타입은 모든 쿼리에서 동일해야 함

유형	설명	도식
UNION	양쪽 쿼리 결과를 모두 포함 (중복 결과는 1번만 표현)	
UNION ALL	양쪽 쿼리 결과를 모두 포함 (중복 결과도 모두 표현)	
INTERSECT	양쪽 쿼리 결과에 모두 포함되는 행만 표현	
MINUS	쿼리1 결과에만 포함되고 쿼리2 결과에는 포함되지 않는 행만 표현	

4.2.5 SET Operator - UNION

SELECT EMP_ID,
 ROLE_NAME
FROM EMPLOYEE_ROLE

UNION

SELECT EMP_ID,
 ROLE_NAME
FROM ROLE_HISTORY;

쿼리1

쿼리2

모든 직원의 현재 ROLE과 이전
ROLE을 함께 표시

[쿼리1 결과]

EMP_ID	ROLE_NAME
100	SALES
101	SALES
102	SALES
103	SE
104	SE
107	SE
124	MKT
141	MKT
143	MKT
144	MKT
149	MKT
174	SALES-MKT
176	SALES-MKT
178	
200	HR
201	MKT
202	HR
205	HR
206	
207	FIN
208	FIN
210	FIN

22건

[쿼리2 결과]

EMP_ID	ROLE_NAME
104	SE-PKG
176	MKT
104	SE-ANLY
104	SE

4건

[UNION 결과]

EMP_ID	ROLE_NAME
100	SALES
101	SALES
102	SALES
103	SE
104	SE
104	SE-ANLY
104	SE-PKG
107	SE
124	MKT
141	MKT
143	MKT
144	MKT
149	MKT
174	SALES-MKT
176	MKT
176	SALES-MKT
178	
200	HR
201	MKT
202	HR
205	HR
206	
207	FIN
208	FIN
210	FIN

25건
중복 1건 제외

4.2.5 SET Operator - UNION ALL

```
SELECT EMP_ID,  
       ROLE_NAME,  
       DEPT_ID  
FROM   EMPLOYEE_ROLE  
  
UNION ALL  
  
SELECT EMP_ID,  
       ROLE_NAME,  
       DEPT_ID  
FROM   ROLE_HISTORY;
```

쿼리1

쿼리2

모든 직원의 현재 ROLE과 이전
ROLE을 중복 값 포함 표시

[쿼리1 결과]

EMP_ID	ROLE_NAME	DEPT_ID
100	SALES	90
101	SALES	90
102	SALES	90
103	SE	60
104	SE	60
107	SE	60
124	MKT	50
141	MKT	50
143	MKT	50
144	MKT	50
149	MKT	50
174	SALES-MKT	80
176	SALES-MKT	80
178		
200	HR	10
201	MKT	50
202	HR	10
205	HR	10
206		
207	FIN	20
208	FIN	20
210	FIN	20

[쿼리2 결과]

EMP_ID	ROLE_NAME	DEPT_ID
104	SE-PKG	90
176	MKT	50
104	SE-ANLY	90
104	SE	60

4건

[UNION ALL 결과]

EMP_ID	ROLE_NAME	DEPT_ID
100	SALES	90
101	SALES	90
102	SALES	90
103	SE	60
104	SE	60
107	SE	60
124	MKT	50
141	MKT	50
143	MKT	50
144	MKT	50
149	MKT	50
174	SALES-MKT	80
176	SALES-MKT	80
178		
200	HR	10
201	MKT	50
202	HR	10
205	HR	10
206		
207	FIN	20
208	FIN	20
210	FIN	20
104	SE-PKG	90
176	MKT	50
104	SE-ANLY	90
104	SE	60

26건
중복 포함

22건

4.2.5 SET Operator - INTERSECT

```
SELECT EMP_ID,  
       ROLE_NAME  
FROM   EMPLOYEE_ROLE  
INTERSECT  
SELECT EMP_ID,  
       ROLE_NAME  
FROM   ROLE_HISTORY;
```

쿼리1

쿼리2

입사 후 현재 ROLE과 동일한
ROLE을 가졌던 적이 있는 직원
조회

[쿼리1 결과]

EMP_ID	ROLE_NAME
100	SALES
101	SALES
102	SALES
103	SE
104	SE
107	SE
124	MKT
141	MKT
143	MKT
144	MKT
149	MKT
174	SALES-MKT
176	SALES-MKT
178	
200	HR
201	MKT
202	HR
205	HR
206	
207	FIN
208	FIN
210	FIN

[쿼리2 결과]

EMP_ID	ROLE_NAME
104	SE-PKG
176	MKT
104	SE-ANLY
104	SE

[INTERSECT 결과]

EMP_ID	ROLE_NAME
104	SE

4.2.5 SET Operator - MINUS

```
SELECT EMP_ID,
       ROLE_NAME
FROM   EMPLOYEE_ROLE
```

쿼리1

MINUS

```
SELECT EMP_ID,
       ROLE_NAME
FROM   ROLE_HISTORY;
```

쿼리2

입사 후 현재 ROLE과 동일한
ROLE을 부여 받은 적이 없는
직원 조회

[쿼리1 결과]

EMP_ID	ROLE_NAME
100	SALES
101	SALES
102	SALES
103	SE
104	SE
107	SE
124	MKT
141	MKT
143	MKT
144	MKT
149	MKT
174	SALES-MKT
176	SALES-MKT
178	
200	HR
201	MKT
202	HR
205	HR
206	
207	FIN
208	FIN
210	FIN

[쿼리2 결과]

EMP_ID	ROLE_NAME
104	SE-PKG
176	MKT
104	SE-ANLY
104	SE

[MINUS 결과]

EMP_ID	ROLE_NAME
100	SALES
101	SALES
102	SALES
103	SE
107	SE
124	MKT
141	MKT
143	MKT
144	MKT
149	MKT
174	SALES-MKT
176	SALES-MKT
178	
200	HR
201	MKT
202	HR
205	HR
206	
207	FIN
208	FIN
210	FIN

4.2.5 SET Operator - 활용

쿼리1과 쿼리2의 SELECT 목록은 반드시 동일(컬럼 개수, 데이터 타입)해야 하므로 이를 위해 Dummy Column을 사용할 수 있다

```
SELECT EMP_NAME,
       JOB_ID,
       HIRE_DATE
FROM   EMPLOYEE
WHERE  DEPT_ID = '20'
```

쿼리1

EMP_NAME	JOB_ID	HIRE_DATE
김술오	J4	96/10/01
이중기	J4	04/10/01
감우섭	J4	05/07/31

UNION

```
SELECT DEPT_NAME,
       DEPT_ID,
       NULL
FROM   DEPARTMENT
WHERE  DEPT_ID = '20';
```

쿼리2

DEPT_NAME	DEPT_ID
회계팀	20

DEPT_NAME	DEPT_ID	NULL
회계팀	20	

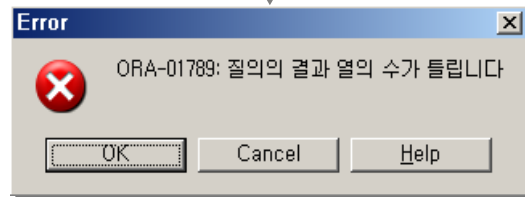
[결과]

EMP_NAME	JOB_ID	HIRE_DATE
감우섭	J4	05/07/31
김술오	J4	96/10/01
이중기	J4	04/10/01
회계팀	20	

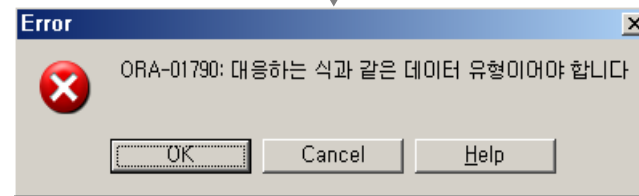
- 쿼리1과 쿼리2는 결과 표시 구조가 동일해야 함
- 구조를 맞추기 위해 해당 위치에 dummy column 사용

4.2.5 SET Operator - 활용

```
SELECT EMP_NAME,  
       JOB_ID,  
       HIRE_DATE  
FROM   EMPLOYEE  
WHERE  DEPT_ID = '20'  
UNION  
SELECT DEPT_NAME,  
       DEPT_ID  
FROM   DEPARTMENT  
WHERE  DEPT_ID = '20';
```



```
SELECT EMP_NAME,  
       SALARY  
FROM   EMPLOYEE  
WHERE  DEPT_ID = '20'  
UNION  
SELECT DEPT_NAME,  
       DEPT_ID  
FROM   DEPARTMENT  
WHERE  DEPT_ID = '20';
```



4.2.5 SET Operator - 활용

실습 : UNION 구문을 이용하여 '50' 부서원을 관리자와 직원을 구분하여 표시하시오

```

SELECT EMP_ID,
       EMP_NAME,
       '관리자' AS 구분
FROM   EMPLOYEE
WHERE  EMP_ID = '141'
AND    DEPT_ID = '50'

UNION

SELECT EMP_ID,
       EMP_NAME,
       '직원' AS 구분
FROM   EMPLOYEE
WHERE  DEPT_ID = '50'
AND    EMP_ID != '141';

```

쿼리1

쿼리2

관리자 정보

EMP_ID	EMP_NAME	구분
141	김예수	관리자

직원 정보

EMP_ID	EMP_NAME	구분
124	정지현	직원
143	나승원	직원
144	김순이	직원
149	성해교	직원
201	박하일	직원

EMP_ID	EMP_NAME	구분
124	정지현	직원
141	김예수	관리자
143	나승원	직원
144	김순이	직원
149	성해교	직원
201	박하일	직원

관리자와 직원 구분을 위해
임의 컬럼을 추가하고
리터럴을 함께 사용

컬럼 별칭은 첫 번째 쿼리에서 지정한 경우에만 적용

4.2.5 SET Operator - 활용

- 기본적으로 첫째 컬럼의 오름차순으로 정렬(UNION ALL은 제외)
- ORDER BY 키워드는 마지막 쿼리 부분에서 한 번만 기술
- 각 쿼리 SELECT 절의 컬럼/별칭 이름이 다르면 ORDER BY 절에 컬럼 기술 순서만 사용

```
SELECT EMP_ID,  
       EMP_NAME,  
       '관리자' AS 구분  
FROM   EMPLOYEE  
WHERE  EMP_ID = '141'  
AND    DEPT_ID = '50'  
UNION  
SELECT EMP_ID,  
       EMP_NAME,  
       '직원' AS 구분  
FROM   EMPLOYEE  
WHERE  DEPT_ID = '50'  
AND    EMP_ID != '141'  
ORDER BY 3, 1;
```



EMP_ID	EMP_NAME	구분
141	김예수	관리자
124	정지현	직원
143	나승원	직원
144	김순이	직원
149	성해교	직원
201	박하일	직원

4.2.5 SET Operator - 활용

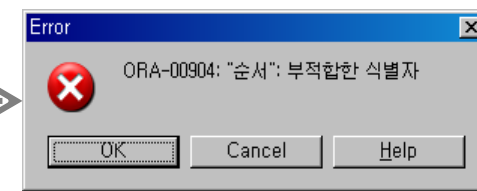
```
SELECT 'SQL을 공부하고 있습니다' 문장, 3 순서 FROM DUAL
UNION
SELECT '우리는 지금', 1 FROM DUAL
UNION
SELECT '아주 재미있게', 2 FROM DUAL;
```

문장	순서
SQL을 공부하고 있습니다	3
아주 재미있게	2
우리는 지금	1

```
SELECT 'SQL을 공부하고 있습니다' 문장, 3 순서 FROM DUAL
UNION
SELECT '우리는 지금', 1 FROM DUAL
UNION
SELECT '아주 재미있게', 2 FROM DUAL
ORDER BY 2;
```

문장	순서
우리는 지금	1
아주 재미있게	2
SQL을 공부하고 있습니다	3

```
SELECT 'SQL을 공부하고 있습니다' 문장, 3 순서 FROM DUAL
UNION
SELECT '우리는 지금', 1 FROM DUAL
UNION
SELECT '아주 재미있게', 2 FROM DUAL
ORDER BY 순서;
```



4.2.5 SET Operator - 활용

SET Operator와 JOIN 관계

```
SELECT EMP_ID,  
       ROLE_NAME  
FROM   EMPLOYEE_ROLE  
JOIN   ROLE_HISTORY USING (EMP_ID, ROLE_NAME);
```

```
SELECT EMP_ID,  
       ROLE_NAME  
FROM   EMPLOYEE_ROLE  
INTERSECT  
SELECT EMP_ID,  
       ROLE_NAME  
FROM   ROLE_HISTORY;
```



EMP_ID	ROLE_NAME
104	SE

4.2.5 SET Operator - 활용

SET Operator와 IN Operator 관계

```
SELECT EMP_NAME,  
       JOB_TITLE 직급  
FROM   EMPLOYEE  
JOIN   JOB USING (JOB_ID)  
WHERE  JOB_TITLE IN ('대리', '사원')  
ORDER BY 2,1;
```

```
SELECT EMP_NAME, '사원' 직급  
FROM   EMPLOYEE  
JOIN   JOB USING (JOB_ID)  
WHERE  JOB_TITLE = '사원'  
UNION  
SELECT EMP_NAME, '대리'  
FROM   EMPLOYEE  
JOIN   JOB USING (JOB_ID)  
WHERE  JOB_TITLE = '대리'  
ORDER BY 2,1;
```



EMP_NAME	직급
권상후	대리
엄정하	대리
임영애	대리
전우성	대리
고승우	사원
성해교	사원
엄정하	사원
정지현	사원

4.2.6 Subquery - 개념

- 하나의 쿼리가 다른 쿼리에 포함되는 구조
- 다른 쿼리에 포함된 내부 쿼리(서브 쿼리)는 외부 쿼리(메인 쿼리)에 사용될 값을 반환하는 역할

요구사항 : '나승원' 직원과 같은 부서원들을 조회하라

요구사항을 해결하기 위해서는

- ❶ '나승원' 직원이 속한 부서가 어떤 부서인지(부서 번호 또는 부서 이름) 찾고
- ❷ 부서 정보(부서 번호 또는 부서 이름)를 이용하여 해당 부서에 속한 직원들을 찾아야 함

메인쿼리

```
SELECT EMP_NAME  
FROM   EMPLOYEE  
WHERE  DEPT_ID = '나승원의 소속부서 ID' ;
```

서브쿼리

```
SELECT DEPT_ID  
FROM   EMPLOYEE  
WHERE  EMP_NAME = '나승원' ;
```

4.2.6 Subquery - 구문

```
SELECT ...  
FROM ...  
WHERE expr operator ( SELECT ...  
                        FROM ...  
                        WHERE ... ) ;
```

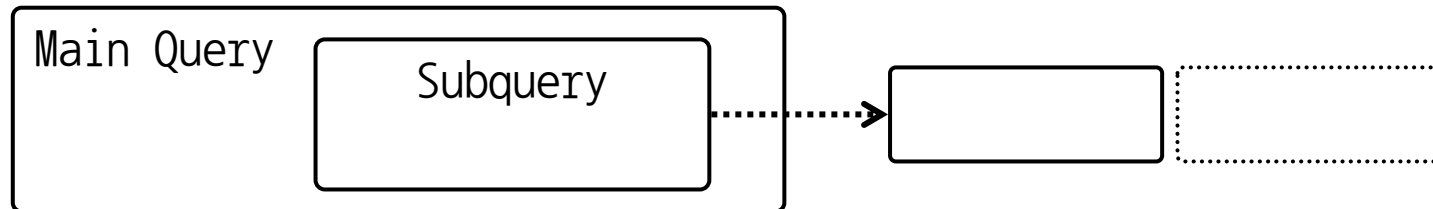
[구문 설명]

- 서브쿼리는 일반적인 SQL 구문과 동일(별도 형식이 존재하는 것이 아님)
- SELECT, FROM, WHERE, HAVING 절 등에서 사용 가능
- 서브쿼리는 ()로 묶어서 표현
- 서브쿼리에는 ; 를 사용하지 않음
- 유형에 따라 연산자를 구분해서 사용

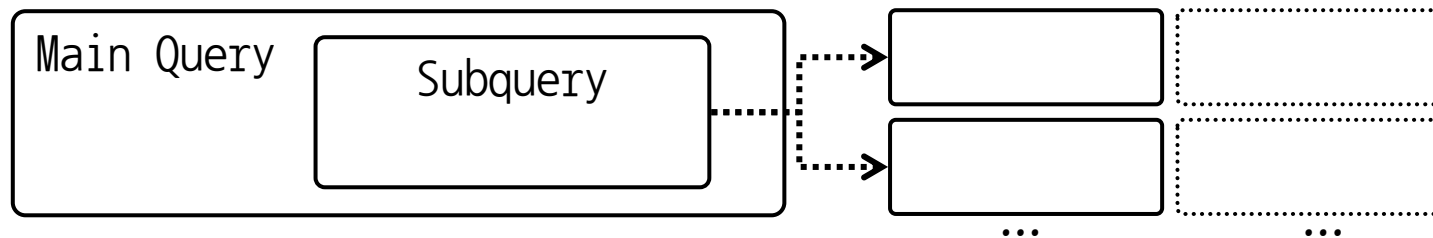
4.2.6 Subquery - 유형

- 단일 행 서브쿼리
 - 단일 행 반환
 - 단일 행 비교 연산자(=, >, >=, <, <=, <> 등) 사용
- 다중 행 서브쿼리
 - 여러 행 반환
 - 다중 행 비교 연산자(IN, ANY, ALL 등) 사용

단일 행 서브쿼리 도식

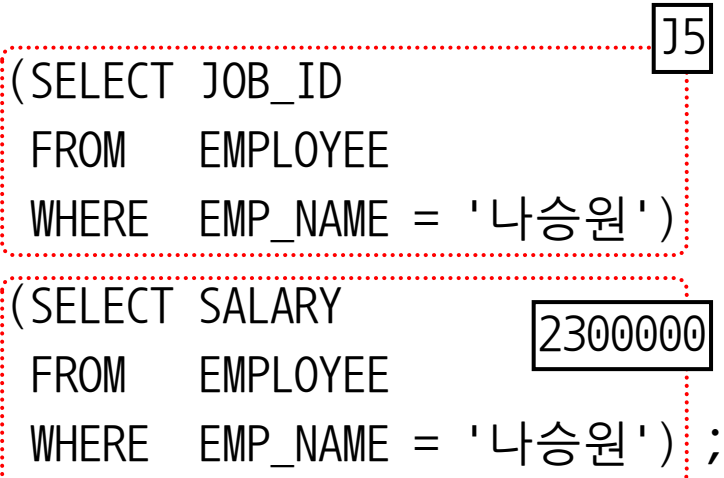


다중 행 서브쿼리 도식



4.2.6 Subquery - 단일 행 서브쿼리

```
SELECT EMP_NAME,  
       JOB_ID,  
       SALARY  
FROM   EMPLOYEE  
WHERE  JOB_ID = (SELECT JOB_ID  
                 FROM   EMPLOYEE  
                 WHERE  EMP_NAME = '나승원')  
  
AND    SALARY > (SELECT SALARY  
                 FROM   EMPLOYEE  
                 WHERE  EMP_NAME = '나승원');
```



```
SELECT EMP_NAME,  
       JOB_ID,  
       SALARY  
FROM   EMPLOYEE  
WHERE  JOB_ID = 'J5'  
AND    SALARY > 23000000;
```



EMP_NAME	JOB_ID	SALARY
박하일	J5	2600000

4.2.6 Subquery - 단일 행 서브쿼리

```
SELECT EMP_NAME,  
       JOB_ID,  
       SALARY  
FROM   EMPLOYEE  
WHERE  SALARY = (SELECT MIN(SALARY)  
                FROM   EMPLOYEE);
```

1500000

```
SELECT EMP_NAME,  
       JOB_ID,  
       SALARY  
FROM   EMPLOYEE  
WHERE  SALARY = 1500000;
```

EMP_NAME	JOB_ID	SALARY
정지현	J7	1500000
고승우	J7	1500000
염정하	J7	1500000

4.2.6 Subquery - 단일 행 서브쿼리

```
SELECT  DEPT_NAME,  
        SUM(SALARY)  
FROM    EMPLOYEE  
LEFT JOIN DEPARTMENT USING (DEPT_ID)  
GROUP BY DEPT_ID, DEPT_NAME  
HAVING  SUM(SALARY) = (SELECT  MAX(SUM(SALARY))  
                       FROM    EMPLOYEE  
                       GROUP BY DEPT_ID);
```

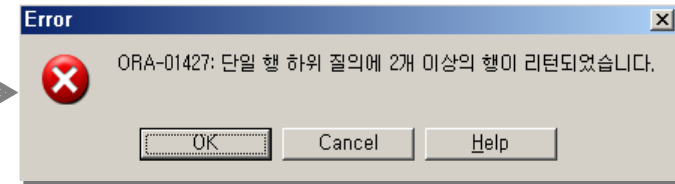
18100000

```
SELECT  DEPT_NAME,  
        SUM(SALARY)  
FROM    EMPLOYEE  
LEFT JOIN DEPARTMENT USING (DEPT_ID)  
GROUP BY DEPT_ID, DEPT_NAME  
HAVING  SUM(SALARY) = 18100000;
```

DEPT_NAME	SUM(SALARY)
해외영업3팀	18100000

4.2.6 Subquery - 단일 행 서브쿼리

```
SELECT  EMP_ID,  
        EMP_NAME  
FROM    EMPLOYEE  
WHERE   SALARY = (SELECT  MIN(SALARY)  
                  FROM    EMPLOYEE  
                  GROUP BY DEPT_ID);
```



MIN(SALARY)
1500000
1500000
2500000
1500000
3600000
2090000
2600000

Subquery 실행 결과 여러 행이 생성되었음
→ 비교 대상 값이 여러 개이므로 오류 발생

4.2.6 Subquery - 다중 행 서브쿼리 ¹⁾IN, NOT IN 연산자

```

SELECT EMP_ID,
       EMP_NAME,
       '관리자' AS 구분
FROM   EMPLOYEE
WHERE  EMP_ID IN (SELECT MGR_ID FROM EMPLOYEE)
UNION
SELECT EMP_ID,
       EMP_NAME,
       '직원'
FROM   EMPLOYEE
WHERE  EMP_ID NOT IN (SELECT MGR_ID FROM EMPLOYEE
                      WHERE MGR_ID IS NOT NULL)

ORDER BY 3, 1;

```

EMP_ID	EMP_NAME	구분
100	한선기	관리자
101	강중훈	관리자
104	안석규	관리자
141	김예수	관리자
174	전우성	관리자
200	고승우	관리자
102	최만식	직원
103	정도연	직원
107	조재형	직원
124	정지현	직원
143	나승원	직원
144	김순이	직원
149	성해교	직원
176	엄정하	직원
178	심하균	직원
201	박하일	직원
202	권상후	직원
205	임영애	직원
206	염정하	직원
207	김술오	직원
208	이중기	직원
210	감우섭	직원

NOT IN 연산자와 다중 행 서브쿼리를 함께 사용하는 경우, 서브쿼리 결과에 NULL이 포함되면 전체 결과가 NULL이 됨

→ 서브쿼리 결과에서 NULL인 경우를 제외시켜야 함

4.2.6 Subquery - 다중 행 서브쿼리 ¹⁾IN, NOT IN 연산자

전체 직원에 대해 관리자와 직원을 구분하여 표시하시오(UNION 구문을 사용하지 말 것)

```
SELECT EMP_ID , EMP_NAME,
       CASE
         WHEN EMP_ID IN (SELECT MGR_ID FROM EMPLOYEE) THEN '관리자'
         ELSE '직원' END AS 구분
FROM   EMPLOYEE
ORDER BY 3, 1;
```

EMP_ID	EMP_NAME	구분
100	한선기	관리자
101	강중훈	관리자
104	안석규	관리자
141	김예수	관리자
174	전우성	관리자
200	고승우	관리자
102	최만식	직원
103	정도연	직원
107	조재형	직원
124	정지현	직원
143	나승원	직원
144	김순이	직원
149	성해교	직원
176	엄정하	직원
178	심하균	직원
201	박하일	직원
202	권상후	직원
205	임영애	직원
206	염정하	직원
207	김술오	직원
208	이중기	직원
210	감우섭	직원

4.2.6 Subquery - 다중 행 서브쿼리 2) ANY 연산자

- < ANY : 비교 대상 중 최대 값 보다 작음
- > ANY : 비교 대상 중 최소 값 보다 큼
- = ANY : IN 연산자와 동일

```

SELECT EMP_NAME,
       SALARY
FROM   EMPLOYEE
JOIN   JOB USING (JOB_ID)
WHERE  JOB_TITLE = '대리'
AND    SALARY > ANY
        (SELECT SALARY
         FROM   EMPLOYEE
         JOIN   JOB USING (JOB_ID)
         WHERE  JOB_TITLE = '과장');

```

EMP_NAME	SALARY
엄정하	2420000
권상후	3410000
임영애	2640000

과장 급여 현황

EMP_NAME	SALARY
김예수	2100000
나승원	2300000
박하일	2600000

대리 급여 현황

EMP_NAME	SALARY
전우성	2090000
엄정하	2420000
권상후	3410000
임영애	2640000

과장 직급보다 급여가 많은 대리 직급 직원 조회

4.2.6 Subquery - 다중 행 서브쿼리 ³⁾ALL 연산자

- < ALL : 비교 대상 중 최소 값 보다 작음
- > ALL : 비교 대상 중 최대 값 보다 큼

```

SELECT EMP_NAME,
       SALARY
FROM   EMPLOYEE
JOIN   JOB USING (JOB_ID)
WHERE  JOB_TITLE = '대리'
AND    SALARY > ALL
        (SELECT SALARY
         FROM   EMPLOYEE
         JOIN   JOB USING (JOB_ID)
         WHERE  JOB_TITLE = '과장');

```

EMP_NAME	SALARY
권상후	3410000
임영애	2640000

과장 급여 현황

EMP_NAME	SALARY
김예수	2100000
나승원	2300000
박하일	2600000

대리 급여 현황

EMP_NAME	SALARY
전우성	2090000
엄정하	2420000
권상후	3410000
임영애	2640000

모든 과장들의 직급보다 급여가 많은 대리 직급 직원 조회

4.2.6 Subquery - 다중 행 서브쿼리

자기 직급의 평균 급여를 받는 직원을 조회하시오

```
SELECT EMP_NAME,
       JOB_TITLE,
       SALARY
FROM   EMPLOYEE
LEFT   JOIN JOB USING (JOB_ID)
WHERE  SALARY IN
       (SELECT TRUNC(AVG(SALARY), -5)
        FROM   EMPLOYEE
        GROUP BY JOB_ID)
ORDER BY JOB_ID;
```

계산 편의를 위해 TRUNC를 사용하였음

EMP_NAME	JOB_TITLE	SALARY
한선기	대표이사	9000000
안석규	부장	3500000
정도연	차장	2600000
이중기	차장	2500000
감우섭	차장	2500000
김술오	차장	2500000
박하일	과장	2600000
나승원	과장	2300000
심하균		2300000

직급 별 평균 급여 현황

JOB_TITLE	AVG
과장	2300000
	2300000
사원	1600000
차장	2500000
대리	2600000
부장	3500000
부사장	4500000
대표이사	9000000

단순히 급여만 비교했기 때문에 다른 직급의 평균
급여와 동일한 경우까지 결과에 포함되었음

→ 직급과 급여를 동시에 비교해야 함

4.2.6 Subquery - 다중 행/다중 열 서브쿼리

자기 직급의 평균 급여를 받는 직원을 조회하시오

```
SELECT EMP_NAME,
       JOB_TITLE,
       SALARY
FROM   EMPLOYEE
LEFT JOIN JOB USING (JOB_ID)
WHERE  (NVL(JOB_ID, ' '), SALARY) IN
      (SELECT NVL(JOB_ID, ' '),
             TRUNC(AVG(SALARY), -5)
       FROM   EMPLOYEE
       GROUP BY JOB_ID)
ORDER BY JOB_ID;
```

EMP_NAME	JOB_TITLE	SALARY
한선기	대표이사	9000000
안석규	부장	3500000
김술오	차장	2500000
감우섭	차장	2500000
이중기	차장	2500000
나승원	과장	2300000
심하균		2300000

직급 별 평균 급여 현황

JOB_TITLE	AVG
과장	2300000
	2300000
사원	1600000
차장	2500000
대리	2600000
부장	3500000
부사장	4500000
대표이사	9000000

4.2.6 Subquery - FROM 절 사용

```

SELECT EMP_NAME,
       JOB_TITLE,
       SALARY
FROM   (SELECT JOB_ID,
               TRUNC(AVG(SALARY), -5) AS JOBAVG
        FROM   EMPLOYEE
        GROUP BY JOB_ID) V
JOIN   EMPLOYEE E ON
       (JOBAVG = SALARY AND
        NVL(E.JOB_ID, ' ') = NVL(V.JOB_ID, ' '))
LEFT  JOIN JOB J ON (E.JOB_ID = J.JOB_ID)
ORDER BY JOB_ID;

```

EMP_NAME	JOB_TITLE	SALARY
한선기	대표이사	9000000
안석규	부장	3500000
김술오	차장	2500000
감우섭	차장	2500000
이중기	차장	2500000
나승원	과장	2300000
심하균		2300000

JOB_ID	JOBAVG
J2	4500000
J7	1600000
	2300000
J3	3500000
J6	2600000
J5	2300000
J1	9000000
J4	2500000

서브쿼리 결과를
테이블처럼 간주

4.2.6 Subquery - Correlated Subquery

메인 쿼리에서 고려된 각 후보 행들에 대해 서브쿼리가 다른 결과를 반환해야 하는 경우(메인 쿼리에서 처리되는 각 행의 값에 따라 응답이 달라져야 하는 경우)에 유용

```
SELECT EMP_NAME,  
       JOB_TITLE,  
       SALARY  
FROM   EMPLOYEE E  
LEFT JOIN JOB J  ON (E.JOB_ID = J.JOB_ID)  
WHERE  SALARY = (SELECT TRUNC(AVG(SALARY), -5)  
                FROM    EMPLOYEE  
                WHERE   NVL(JOB_ID, ' ') =  
                        NVL(E.JOB_ID, ' '))  
ORDER BY E.JOB_ID;
```

EMP_NAME	JOB_TITLE	SALARY
한선기	대표이사	9000000
안석규	부장	3500000
김술오	차장	2500000
이중기	차장	2500000
감우섭	차장	2500000
나승원	과장	2300000
심하균		2300000

- 서브쿼리의 WHERE 절 수행을 위해서는 메인쿼리가 먼저 수행되는 구조
- 메인쿼리 행 수 만큼 반복적으로 수행됨

4.2.6 Subquery - EXISTS, NOT EXISTS 연산자

- 결과에 포함되는지의 여부를 확인해야 하는 경우에 유용한 연산자
- 존재 여부에 따라 TRUE 값을 반환

메인쿼리 한 행을
읽고 해당 값을
서브쿼리에서
참조하여 서브쿼리
결과에 존재하면
TRUE 반환

메인쿼리 한 행을
읽고 해당 값을
서브쿼리에서
참조하여 서브쿼리
결과에 존재하지
않으면 TRUE 반환

```
SELECT EMP_ID, EMP_NAME, '관리자' AS 구분
FROM   EMPLOYEE E
WHERE  EXISTS (SELECT NULL
                FROM   EMPLOYEE
                WHERE  E.EMP_ID = MGR_ID)

UNION

SELECT EMP_ID, EMP_NAME, '직원'
FROM   EMPLOYEE E2
WHERE  NOT EXISTS (SELECT NULL
                   FROM   EMPLOYEE
                   WHERE  E2.EMP_ID = MGR_ID)

ORDER BY 3, 1;
```

EMP_ID	EMP_NAME	구분
100	한선기	관리자
101	강중훈	관리자
104	안석규	관리자
141	김예수	관리자
174	전우성	관리자
200	고승우	관리자
102	최만식	직원
103	정도연	직원
107	조재형	직원
124	정지현	직원
143	나승원	직원
144	김순이	직원
149	성해교	직원
176	엄정하	직원
178	심하균	직원
201	박하일	직원
202	권상후	직원
205	임영애	직원
206	엄정하	직원
207	김술오	직원
208	이중기	직원
210	감우섭	직원

참고 존재 여부를 확인하는 목적이므로 서브쿼리에서 특정 값을 조회할 필요가 없음